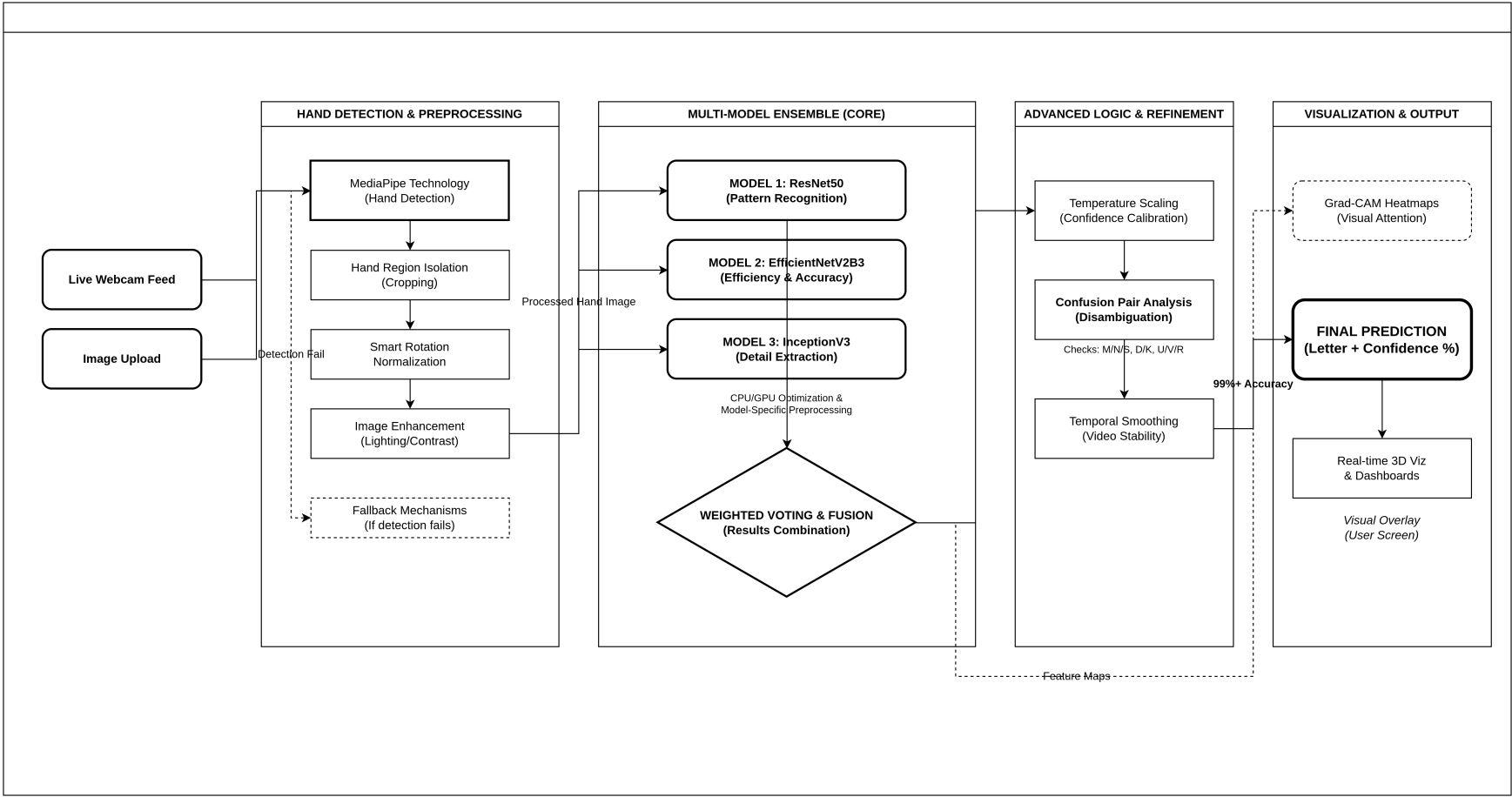




1. Multi-Model Ensemble Approach

- Uses three different AI models (ResNet50, EfficientNetV2B3, InceptionV3) that complement each other
- Each model has different strengths in recognizing hand shapes and patterns
- Results are combined using weighted voting for more reliable predictions
- Models were trained on thousands of hand sign images with MediaPipe preprocessing

2. Hand Detection & Preprocessing

- Advanced hand detection using MediaPipe technology
- Smart rotation normalization to handle hands in different orientations
- Multiple fallback mechanisms when detection fails
- Image enhancement techniques to improve quality under various lighting conditions

3. Interactive Visualization

- Real-time 3D visualizations showing model confidence and decision-making
- Heatmaps highlighting which parts of the hand the AI is focusing on
- Comprehensive metrics dashboards showing performance across all letters
- Interactive charts for exploring model performance

How It Works

For Image Upload:

1. User uploads or takes a photo containing a hand sign
2. System detects and isolates the hand region
3. Multiple AI models analyze the hand position simultaneously
4. Results are combined with confidence scoring
5. Visual overlay shows the prediction with attention heatmaps
6. System explains potentially confusing letters (like M/N/S) with contextual analysis

For Live Camera:

1. Accesses webcam feed in real-time
2. Continuously detects hands and applies preprocessing
3. Runs ensemble prediction with temporal smoothing for stability
4. Displays prediction with confidence meter
5. Uses model fusion to achieve 99%+ accuracy while maintaining speed

Key Innovations

Accuracy Enhancements:

- Temperature scaling for better confidence calibration
- Specialized ensemble weights based on research
- Confusion pair analysis for commonly mixed-up letters (M/N/S, D/K, U/V/R)
- Grad-CAM visual explanations showing model attention

Performance Optimizations:

- Intelligent frame skipping for smooth video processing
- CPU/GPU optimization to work on various devices
- Model-specific preprocessing to match training conditions
- Temporal smoothing for stable predictions in video

Practical Features

- No installation required - runs directly in web browser
- Works with any webcam or uploaded image
- Multiple operational modes (high accuracy vs. high speed)
- Detailed explanations when confidence is low
- Visual feedback showing exactly what the AI is "seeing"
- Comprehensive performance statistics for educational purposes

1. ✓ Installation and Setup

```
import streamlit as st
import cv2
import numpy as np
import json
import os
import tempfile
from collections import Counter
import threading
from collections import deque
import logging
from typing import Tuple, Optional, Dict, List
from types import SimpleNamespace
import time
import urllib.request
from streamlit_webrtc import webrtc_streamer, VideoTransformerBase, WebRtcMode
import av
import asyncio

# Force CPU mode
os.environ['CUDA_VISIBLE_DEVICES'] = '-1'

# Configure logging
if not logging.getLogger(__name__).handlers:
    logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Set a gentle asyncio exception handler to reduce noisy aioice callback logs
def _asyncio_exception_handler(loop, context):
    try:
        msg = context.get('message') or ''
        exc = context.get('exception')
        # Filter a known benign aioice retry/transport message if it appears
        if isinstance(exc, AttributeError) and "NoneType" object has no attribute 'sendto'
    in str(exc):
        logger.debug(f"Suppressed asyncio/aioice AttributeError: {exc}")
        return
    logger.error(f"Asyncio exception: {msg} - {exc}")
    except Exception:
        # keep exception handler itself safe
        logger.exception("Error in asyncio exception handler")

try:
    loop = asyncio.get_event_loop()
    loop.set_exception_handler(_asyncio_exception_handler)
except Exception:
    # If the streamlit-hosted event loop isn't available yet, skip setting handler
    logger.debug("Could not set asyncio exception handler at import time")

try:
    import tensorflow as tf
    import mediapipe as mp
    from huggingface_hub import hf_hub_download
except ImportError as e:
    st.error(f"Missing dependency: {e}")
    st.stop()
```

2. ✓ Page Configuration and Constants

```
# CONFIGURATION
st.set_page_config(
    page_title="ASL Multi-Model Classifier",
    page_icon=None,
    layout="wide",
    initial_sidebar_state="expanded"
)

IMG_SIZE = 224

# Hugging Face Space that hosts the trained ASL models (.keras files)
# This is the space you shared: ALI-ezz/train_asl_models-5
REPO_ID = "ALI-ezz/train_asl_models-5"

# Crop padding factor applied to landmark bbox (can be tuned)
PAD_FACTOR = 1.4

# ACCURACY IMPROVEMENT CONFIGURATION
# Temperature scaling: Higher = softer probabilities, more calibrated confidence
# Typical values: 1.0 (no scaling) to 3.0 (very soft)
DEFAULT_TEMPERATURE = 1.5

# Improved ensemble weights based on research (EfficientNet typically best for ASL)
DEFAULT_ENSEMBLE_WEIGHTS = {
    'EfficientNetV2B3': 0.45, # Highest - typically best accuracy
    'ResNet50': 0.30, # Good for general features
    'InceptionV3': 0.25 # Good for multi-scale features
}

# Commonly confused letter pairs in ASL (for post-processing)
CONFUSION_PAIRS = {
    'M': ['N', 'S', 'A'],
    'N': ['M', 'S', 'A'],
    'S': ['M', 'N', 'A', 'E'],
    'A': ['S', 'E', 'M', 'N'],
    'E': ['A', 'S'],
    'D': ['K', 'F'],
    'K': ['D', 'P', 'V'],
    'U': ['V', 'R', 'H'],
    'V': ['U', 'K', 'R'],
    'R': ['U', 'V'],
    'P': ['K', 'Q'],
    'Q': ['P'],
    'H': ['U', 'G'],
    'G': ['H', 'Q'],
    'I': ['J', 'Y'],
    'J': ['I'],
    'Y': ['I'],
    'L': ['F'],
    'F': ['L', 'D'],
}
```

3. ✓ Enhanced Preprocessing Functions

```
# ENHANCED PREPROCESSING
def enhance_image(image_rgb: np.ndarray) -> np.ndarray:
    """Apply contrast enhancement for better detection"""
    # Convert to LAB color space
    lab = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    # Apply CLAHE to L channel
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
    l = clahe.apply(l)
    # Merge and convert back
    enhanced_lab = cv2.merge([l, a, b])
    enhanced_rgb = cv2.cvtColor(enhanced_lab, cv2.COLOR_LAB2RGB)
    return enhanced_rgb

def grayworld_color_correction(img: np.ndarray) -> np.ndarray:
    """Simple Gray-World color constancy: scale channels so their means match overall mean."""
    try:
        img_f = img.astype(np.float32)
        means = img_f.mean(axis=(0,1))
        if np.any(means <= 0):
            return img
        mean_gray = means.mean()
        scale = mean_gray / (means + 1e-9)
        img_corr = img_f * scale[np.newaxis, np.newaxis, :]
        img_corr = np.clip(img_corr, 0, 255).astype(np.uint8)
        return img_corr
    except Exception:
        return img

def gamma_correction(img: np.ndarray, target_mean: float = 128.0) -> np.ndarray:
    """Apply simple gamma correction to normalize image brightness toward a target mean.
    This helps very dark or very bright frames reach a more middle exposure for MediaPipe.
    """
    try:
        img_f = img.astype(np.float32)
        mean = img_f.mean()
        if mean <= 1.0:
            return img
        # compute gamma: if mean < target -> gamma < 1 makes image brighter when using pow
        # we compute ratio and clamp gamma
        ratio = float(target_mean) / (mean + 1e-9)
        # map ratio to gamma in a stable way
        gamma = np.clip(-np.log2(ratio) + 1.0, 0.4, 2.5)
        invGamma = 1.0 / gamma
        table = np.array([(i / 255.0) ** invGamma) * 255 for i in
np.arange(256)]).astype("uint8")
        corrected = cv2.LUT(img.astype(np.uint8), table)
        return corrected
    except Exception:
        return img

def _attach_center(landmarks_obj, bbox, frame_shape: Tuple[int, int]):
    """Compute and attach a normalized center (x,y in 0..1) to a landmarks object.
    Args:
        landmarks_obj: object with attribute landmark (iterable of objects with .x and .y),
or None.
        bbox: tuple (x1,y1,x2,y2) in pixel coords, or None.
        frame_shape: (height, width) of the image the landmarks relate to.
    Returns:
        landmarks_obj (possibly synthetic) with attribute .center set to
SimpleNamespace(x=..., y=...).
    """
    try:
        h_orig, w_orig = frame_shape
        pts = getattr(landmarks_obj, 'landmark', None)
        if not pts:
            if bbox is None:
                cx = 0.5
                cy = 0.5
            else:
                x1, y1, x2, y2 = bbox
```

```

        cx = ((x1 + x2) / 2.0) / float(w_orig)
        cy = ((y1 + y2) / 2.0) / float(h_orig)
        synthetic = SimpleNamespace(landmark=[SimpleNamespace(x=float(cx),
y=float(cy))])
        synthetic.center = SimpleNamespace(x=float(cx), y=float(cy))
        return synthetic
    xs = [float(getattr(p, 'x', 0.0)) for p in pts if getattr(p, 'x', None) is not
None]
    ys = [float(getattr(p, 'y', 0.0)) for p in pts if getattr(p, 'y', None) is not
None]
    if len(xs) > 0 and len(ys) > 0:
        cx = float(np.mean(xs))
        cy = float(np.mean(ys))
    else:
        if bbox is not None:
            x1, y1, x2, y2 = bbox
            cx = ((x1 + x2) / 2.0) / float(w_orig)
            cy = ((y1 + y2) / 2.0) / float(h_orig)
        else:
            cx = 0.5
            cy = 0.5
    try:
        landmarks_obj.center = SimpleNamespace(x=float(cx), y=float(cy))
    except Exception:
        pass
    return landmarks_obj
except Exception:
    return landmarks_obj

```


4. ✓ Additional Lightweight Fallbacks for Hand Detection

```
# Additional lightweight fallbacks (no new ML model)
def _template_match_proposal(frame_rgb: np.ndarray, min_score: float = 0.6) -> List[tuple]:
    """Try to locate previous hand template in the current frame via template matching.
    Returns a list of proposal bboxes (x1,y1,x2,y2) or empty list.
    """
    proposals = []
    try:
        templ = st.session_state.get('prev_hand_template', None)
        if templ is None:
            return proposals
        # convert to gray for matching
        gray = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2GRAY)
        templ_gray = cv2.cvtColor(np.array(templ, dtype=np.uint8), cv2.COLOR_RGB2GRAY)
        th, tw = templ_gray.shape[:2]
        # if template larger than frame, skip
        h, w = gray.shape[:2]
        if th >= h or tw >= w:
            return proposals
        res = cv2.matchTemplate(gray, templ_gray, cv2.TM_CCOEFF_NORMED)
        min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)
        if max_val >= min_score:
            x1, y1 = max_loc
            x2, y2 = x1 + tw, y1 + th
            # expand a little context
            pad = int(0.2 * max(tw, th))
            x1 = max(0, x1 - pad)
            y1 = max(0, y1 - pad)
            x2 = min(w, x2 + pad)
            y2 = min(h, y2 + pad)
            proposals.append((x1, y1, x2, y2))
    except Exception:
        pass
    return proposals

def _motion_based_proposals(frame_rgb: np.ndarray, min_area: int = 500) -> List[tuple]:
    """Generate proposals from frame-to-frame motion (absdiff). Requires `prev_frame_gray`
    in session state.
    Useful for video/realtime to find moving hands (no extra model needed).
    """
    proposals = []
    try:
        prev = st.session_state.get('prev_frame_gray', None)
        gray = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2GRAY)
        if prev is None:
            # store current and return none
            st.session_state['prev_frame_gray'] = gray
            return proposals
        diff = cv2.absdiff(prev, gray)
        _, th = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7,7))
        th = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel, iterations=1)
        th = cv2.morphologyEx(th, cv2.MORPH_CLOSE, kernel, iterations=1)
        contours, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        h, w = gray.shape[:2]
        for c in contours:
            x, y, cw, ch = cv2.boundingRect(c)
            if cw * ch < min_area:
                continue
            pad_px = int(0.2 * max(cw, ch))
            x1 = max(0, x - pad_px)
            y1 = max(0, y - pad_px)
            x2 = min(w, x + cw + pad_px)
            y2 = min(h, y + ch + pad_px)
            proposals.append((x1, y1, x2, y2))
        # update prev frame for next iteration
        st.session_state['prev_frame_gray'] = gray
    except Exception:
        pass
    return proposals
```


5. ✓ Model Loading and Management

```
# MODEL LOADING
@st.cache_resource
def download_and_load_models():
    """Download and load models with timeout handling"""
    try:
        with open('class_names.json', 'r') as f:
            classes = json.load(f)
    except:
        classes = [f"Class_{i}" for i in range(29)]

    models = {}
    model_configs = {
        'EfficientNetV2B3': 'EfficientNetV2B3_best.keras', # Load best model first
        'InceptionV3': 'InceptionV3_best.keras',
        'ResNet50': 'ResNet50_best.keras',
    }

    for model_name, filename in model_configs.items():
        try:
            # Add timeout for download
            import socket
            old_timeout = socket.setdefaulttimeout()
            socket.setdefaulttimeout(30) # 30 second timeout
            try:
                hf_token = st.secrets.get('HF_TOKEN', None)
                model_path = hf_hub_download(
                    repo_id=REPO_ID,
                    filename=filename,
                    repo_type="space",
                    cache_dir=tempfile.gettempdir(),
                    local_files_only=False,
                    token=hf_token
                )
            finally:
                socket.setdefaulttimeout(old_timeout)

            try:
                # Try to load the full model first
                model = tf.keras.models.load_model(model_path)
                models[model_name] = model
            except Exception as load_err:
                logger.warning(f"Direct load failed for {model_name}: {load_err}. Trying
weights-only load into constructed architecture.")
                # Attempt to construct matching architecture and load weights
                try:
                    if model_name == 'ResNet50':
                        base = tf.keras.applications.ResNet50(weights='imagenet',
include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
                        top = tf.keras.Sequential([
                            base,
                            tf.keras.layers.GlobalAveragePooling2D(),
                            tf.keras.layers.BatchNormalization(),
                            tf.keras.layers.Dropout(0.5),
                            tf.keras.layers.Dense(512, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)),
                            tf.keras.layers.BatchNormalization(),
                            tf.keras.layers.Dropout(0.4),
                            tf.keras.layers.Dense(256, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)),
                            tf.keras.layers.Dropout(0.3),
                            tf.keras.layers.Dense(len(classes), activation='softmax')
                        ], name='ResNet50_AS_L')
                        model = top
                    elif model_name == 'InceptionV3':
                        base = tf.keras.applications.InceptionV3(weights='imagenet',
include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
                        top = tf.keras.Sequential([
                            base,
                            tf.keras.layers.GlobalAveragePooling2D(),
                            tf.keras.layers.BatchNormalization(),
                            tf.keras.layers.Dropout(0.45),
                            tf.keras.layers.Dense(512, activation='relu',
```



```

kernel_regularizer=tf.keras.regularizers.l2(0.001)),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Dropout(0.35),
        tf.keras.layers.Dense(256, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)),
        tf.keras.layers.Dropout(0.25),
        tf.keras.layers.Dense(len(classes), activation='softmax')
    ], name='InceptionV3_ASL')
    model = top
    elif model_name == 'EfficientNetV2B3':
        try:
            base =
tf.keras.applications.EfficientNetV2B3(weights='imagenet', include_top=False, input_shape=
(IMG_SIZE, IMG_SIZE, 3))
        except Exception:
            base = tf.keras.applications.EfficientNetV2B3(weights=None,
include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
            top = tf.keras.Sequential([
                base,
                tf.keras.layers.GlobalAveragePooling2D(),
                tf.keras.layers.BatchNormalization(),
                tf.keras.layers.Dropout(0.5),
                tf.keras.layers.Dense(512, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)),
                tf.keras.layers.BatchNormalization(),
                tf.keras.layers.Dropout(0.35),
                tf.keras.layers.Dense(256, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.001)),
                tf.keras.layers.Dropout(0.25),
                tf.keras.layers.Dense(len(classes), activation='softmax')
            ], name='EfficientNetV2B3_ASL')
            model = top
        else:
            raise RuntimeError(f"No builder available for {model_name}")

    # Try loading weights from the .keras file into the constructed model
    try:
        model.load_weights(model_path)
        models[model_name] = model
        logger.info(f"Loaded weights into constructed {model_name} model
from {model_path}")
    except Exception as werr:
        logger.warning(f"Weights load failed for {model_name}: {werr}.
Trying by_name=True")
        try:
            model.load_weights(model_path, by_name=True)
            models[model_name] = model
            logger.info(f"Loaded weights by_name into constructed
{model_name} model from {model_path}")
        except Exception as werr2:
            logger.error(f"Failed to load weights for {model_name}:
{werr2}")
    except Exception as build_err:
        logger.error(f"Failed to build+load weights for {model_name}:
{build_err}")
    except Exception as e:
        logger.error(f"Failed to load {model_name}: {e}")

    if len(models) == 0:
        st.error("No models loaded!")
        st.stop()

    return models, classes

with st.spinner("Loading AI models..."):
    try:
        MODELS, CLASS_NAMES = download_and_load_models()
        # Show warning if EfficientNetV2B3 failed to load
        if 'EfficientNetV2B3' not in MODELS:
            st.warning("EfficientNetV2B3 failed to load - check logs for details")
    except Exception as e:
        st.error(f"Failed to load models: {e}")
        st.stop()

```

6. ✓ Runtime Toggles and Configuration

```
# Lightweight runtime toggles
st.session_state.setdefault('debug_verbose', False)
st.session_state.setdefault('fast_mode', True)
st.session_state.setdefault('fast_max_width', 640)
```

7. ✓ Hand Detection with MediaPipe (CRITICAL FIX)

```
# HAND DETECTION - CRITICAL FIX
def detect_hand_in_frame(frame_rgb: np.ndarray,
                        detector: mp.solutions.hands.Hands,
                        enhance: bool = True,
                        allow_fallback: bool = True,
                        ) -> Tuple[Optional[np.ndarray], Optional[tuple], Optional[any],
bool]:
    """
    CRITICAL FIX: Proper hand detection with enhancement
    Args:
        frame_rgb: RGB image (NOT BGR!)
        detector: MediaPipe Hands detector
        enhance: Apply contrast enhancement
    Returns:
        (hand_crop_rgb, bbox, landmarks) or (None, None, None)
    """
    if frame_rgb is None or frame_rgb.size == 0:
        return None, None, None, False

    # Fast-mode: optionally downscale the frame for quicker detection (keeps normalized
    coords)
    try:
        h_orig, w_orig = frame_rgb.shape[:2]
    except Exception:
        h_orig, w_orig = (0, 0)

    fast_mode = False
    try:
        fast_mode = bool(st.session_state.get('fast_mode', False))
    except Exception:
        fast_mode = False

    proc_img = frame_rgb
    proc_scale = 1.0
    try:
        if fast_mode and w_orig > int(st.session_state.get('fast_max_width', 640)):
            target_w = int(st.session_state.get('fast_max_width', 640))
            target_h = int(h_orig * (target_w / float(w_orig))) if w_orig > 0 else h_orig
            proc_img = cv2.resize(frame_rgb, (target_w, max(1, target_h)))
            proc_scale = float(target_w) / float(max(1, w_orig))
    except Exception:
        proc_img = frame_rgb
        proc_scale = 1.0

    # If user provided manual/partial landmarks via session state, use them
    try:
        manual = st.session_state.get('manual_landmarks', None)
        if manual:
            # manual expected as list of dicts or tuples: [{'x':0.5,'y':0.4}, ...] or
            [(x,y), ...]
            h_orig, w_orig = frame_rgb.shape[:2]
            pts = []
            for p in manual:
                if isinstance(p, dict):
                    x = float(p.get('x', 0.0))
                    y = float(p.get('y', 0.0))
                elif isinstance(p, (list, tuple)) and len(p) >= 2:
                    x, y = float(p[0]), float(p[1])
                else:
                    continue
            # If coordinates look like pixel coords, convert to normalized
            if x > 1.0 or y > 1.0:
                xn = x / float(w_orig)
                yn = y / float(h_orig)
            else:
                xn = x
                yn = y
            pts.append(SimpleNamespace(x=xn, y=yn))
            if len(pts) > 0:
                # Compute bbox from provided points
                x_coords = [p.x for p in pts]
                y_coords = [p.y for p in pts]
```

```

x_min = min(x_coords)
x_max = max(x_coords)
y_min = min(y_coords)
y_max = max(y_coords)
# If only one point, expand a reasonable area around it
if len(pts) == 1:
    side_frac = float(st.session_state.get('manual_point_side', 0.18))
    cx = x_coords[0]
    cy = y_coords[0]
    side = side_frac
else:
    cx = (x_min + x_max) / 2.0
    cy = (y_min + y_max) / 2.0
    side = max(x_max - x_min, y_max - y_min) *
float(st.session_state.get('pad_factor', PAD_FACTOR))

# Map to pixels
x1 = int(max(0, (cx - side/2) * w_orig))
y1 = int(max(0, (cy - side/2) * h_orig))
x2 = int(min(w_orig, (cx + side/2) * w_orig))
y2 = int(min(h_orig, (cy + side/2) * h_orig))

if x2 > x1 and y2 > y1:
    crop = frame_rgb[y1:y2, x1:x2]
    # Build synthetic landmarks object compatible enough for downstream use
    synthetic = SimpleNamespace(landmark=pts)
    try:
        tpl = cv2.resize(crop, (int(min(160, crop.shape[1])), int(min(160,
crop.shape[0]))))
        st.session_state['prev_hand_template'] = tpl.tolist()
    except Exception:
        pass
    synthetic = _attach_center(synthetic, (x1, y1, x2, y2),
frame_rgb.shape[:2])
    return crop, (x1, y1, x2, y2), synthetic, False
except Exception:
    pass

# We'll try multiple variants to improve robustness:
# 1) enhanced image (CLAHE)
# 2) original image
# 3) horizontally flipped image (useful for mirrored hands)
# 4) upscaled images (1.25x, 1.5x) to help detect small hands
variants = []
try:
    if enhance:
        variants.append({'img': enhance_image(frame_rgb), 'scale': 1.0, 'flip': False,
'desc': 'enhanced'})
        variants.append({'img': frame_rgb, 'scale': 1.0, 'flip': False, 'desc':
'original'})
        # Gray-world corrected variant to address color casts
        try:
            gw = grayworld_color_correction(frame_rgb)
            variants.append({'img': gw, 'scale': 1.0, 'flip': False, 'desc': 'grayworld'})
        except Exception:
            pass
        # flipped
        try:
            flipped = cv2.flip(frame_rgb, 1)
            variants.append({'img': flipped, 'scale': 1.0, 'flip': True, 'desc':
'flipped'})
        except Exception:
            pass
        # gamma-corrected variant (optional toggle via session_state)
        try:
            if st.session_state.get('enable_gamma_correction', False):
                g = gamma_correction(frame_rgb)
                variants.append({'img': g, 'scale': 1.0, 'flip': False, 'desc': 'gamma'})
        except Exception:
            pass
        # upscales to help detect small hands
        h0, w0 = frame_rgb.shape[:2]
        for s in (1.25, 1.5):
            try:
                up = cv2.resize(frame_rgb, (int(w0 * s), int(h0 * s)))

```

```

        variants.append({'img': up, 'scale': s, 'flip': False, 'desc':
f'up{int(s*100)}'})
    except Exception:
        pass
except Exception as e:
    logger.debug(f"Preparing detection variants failed: {e}")

# Try each variant until we get a detection
for var in variants:
    img = var['img']
    scale = var.get('scale', 1.0)
    flip = var.get('flip', False)
    try:
        results = detector.process(img)
    except Exception as e:
        logger.debug(f"MediaPipe process error on variant {var.get('desc')}: {e}")
        continue

    if not results or not results.multi_hand_landmarks:
        continue

    hand_landmarks = results.multi_hand_landmarks[0]
    # If we used a flipped image, mirror the x coords so landmarks map to original
    if flip:
        try:
            for lm in hand_landmarks.landmark:
                lm.x = 1.0 - lm.x
        except Exception:
            pass

    # Calculate normalized bbox from landmarks (normalized coords 0..1)
    x_coords = [lm.x for lm in hand_landmarks.landmark]
    y_coords = [lm.y for lm in hand_landmarks.landmark]
    x_min = min(x_coords)
    x_max = max(x_coords)
    y_min = min(y_coords)
    y_max = max(y_coords)

    # Use PAD_FACTOR padding (tunable)
    pad = PAD_FACTOR
    try:
        pad = float(st.session_state.get('pad_factor', PAD_FACTOR))
    except Exception:
        pad = PAD_FACTOR

    cx = (x_min + x_max) / 2
    cy = (y_min + y_max) / 2
    side = max(x_max - x_min, y_max - y_min) * pad

    # Map normalized coords back to original frame pixels
    # Note: landmarks normalized coords are relative to the processed image size.
    # If we scaled the image by `scale`, the normalized coords remain consistent
    # (fractions), so mapping to original uses original width/height directly.
    h_orig, w_orig = frame_rgb.shape[:2]
    x1 = int(max(0, (cx - side/2) * w_orig))
    y1 = int(max(0, (cy - side/2) * h_orig))
    x2 = int(min(w_orig, (cx + side/2) * w_orig))
    y2 = int(min(h_orig, (cy + side/2) * h_orig))

    # Extract crop from ORIGINAL image (not the variant image)
    if x2 > x1 and y2 > y1:
        cropped = frame_rgb[y1:y2, x1:x2]
        if cropped.size > 0 and min(cropped.shape[:2]) > 20:
            # store small template for template-tracking fallback
            try:
                tpl = cv2.resize(cropped, (int(min(160, cropped.shape[1])),
int(min(160, cropped.shape[0]))))
                st.session_state['prev_hand_template'] = tpl.tolist()
            except Exception:
                pass

        hand_landmarks = _attach_center(hand_landmarks, (x1, y1, x2, y2),
frame_rgb.shape[:2])
        return cropped, (x1, y1, x2, y2), hand_landmarks, False

```

```

# If no detection yet, try targeted proposals using skin-color segmentation
try:
    h, w = frame_rgb.shape[:2]
    proposals = []
    # Try HSV-based skin mask with two ranges (broad and narrow)
    hsv = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2HSV)
    ranges = [
        (np.array([0, 15, 40], dtype=np.uint8), np.array([35, 255, 255],
dtype=np.uint8)),
        (np.array([0, 30, 60], dtype=np.uint8), np.array([25, 200, 255],
dtype=np.uint8))
    ]
    for lo, hi in ranges:
        try:
            mask = cv2.inRange(hsv, lo, hi)
            kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7,7))
            mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)
            mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
            contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
            for c in contours:
                x, y, cw, ch = cv2.boundingRect(c)
                area = cw * ch
                if area < 400: # too small
                    continue
                pad_px = int(0.2 * max(cw, ch))
                x1 = max(0, x - pad_px)
                y1 = max(0, y - pad_px)
                x2 = min(w, x + cw + pad_px)
                y2 = min(h, y + ch + pad_px)
                proposals.append((x1, y1, x2, y2))
        except Exception:
            pass
    # Try YCrCb-based skin detector as alternative
    try:
        ycrb = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2YCrCb)
        cr = ycrb[:, :, 1]
        cb = ycrb[:, :, 2]
        mask2 = cv2.inRange(ycrb, np.array((0,135,85), dtype=np.uint8),
np.array((255,180,135), dtype=np.uint8))
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7,7))
        mask2 = cv2.morphologyEx(mask2, cv2.MORPH_OPEN, kernel, iterations=2)
        contours2, _ = cv2.findContours(mask2, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        for c in contours2:
            x, y, cw, ch = cv2.boundingRect(c)
            if cw * ch < 400:
                continue
            pad_px = int(0.2 * max(cw, ch))
            x1 = max(0, x - pad_px)
            y1 = max(0, y - pad_px)
            x2 = min(w, x + cw + pad_px)
            y2 = min(h, y + ch + pad_px)
            proposals.append((x1, y1, x2, y2))
    except Exception:
        pass
    # Edge-based proposals (Canny) to catch high-contrast silhouettes (gloved or dark
hands)
    try:
        gray = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2GRAY)
        blur = cv2.GaussianBlur(gray, (5,5), 0)
        edges = cv2.Canny(blur, 30, 100)
        kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5,5))
        edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=1)
        contours_e, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        for c in contours_e:
            x, y, cw, ch = cv2.boundingRect(c)
            if cw * ch < 400:
                continue
            pad_px = int(0.15 * max(cw, ch))
            x1 = max(0, x - pad_px)
            y1 = max(0, y - pad_px)
            x2 = min(w, x + cw + pad_px)
            y2 = min(h, y + ch + pad_px)

```



```

        proposals.append((x1, y1, x2, y2))
    except Exception:
        pass
# Optionally add template-match and motion-based proposals (video/realtime)
try:
    if st.session_state.get('enable_template_tracking', False):
        try:
            tpl = _template_match_proposal(frame_rgb)
            for p in tpl:
                proposals.append(p)
        except Exception:
            pass
    if st.session_state.get('enable_motion_proposals', False):
        try:
            mprops = _motion_based_proposals(frame_rgb)
            for p in mprops:
                proposals.append(p)
        except Exception:
            pass
except Exception:
    pass
# Deduplicate proposals
uniq = []
for p in proposals:
    if p not in uniq:
        uniq.append(p)
proposals = uniq

# Try MediaPipe on each proposal region (smaller images, faster)
for (x1, y1, x2, y2) in proposals:
    try:
        crop = frame_rgb[y1:y2, x1:x2]
        if crop.size == 0:
            continue
        # run detector on crop
        res = detector.process(crop)
        if res and res.multi_hand_landmarks:
            lm = res.multi_hand_landmarks[0]
            # Map crop-relative landmarks back to normalized original coords
            ch, cw = crop.shape[:2]
            h_orig, w_orig = frame_rgb.shape[:2]
            for point in lm.landmark:
                point.x = (x1 + point.x * cw) / float(w_orig)
                point.y = (y1 + point.y * ch) / float(h_orig)
            # build bbox from mapped landmarks
            x_coords = [p.x for p in lm.landmark]
            y_coords = [p.y for p in lm.landmark]
            cx = (min(x_coords) + max(x_coords)) / 2
            cy = (min(y_coords) + max(y_coords)) / 2
            side = max(max(x_coords)-min(x_coords), max(y_coords)-min(y_coords)) *
pad
            x1n = int(max(0, (cx - side/2) * w_orig))
            y1n = int(max(0, (cy - side/2) * h_orig))
            x2n = int(min(w_orig, (cx + side/2) * w_orig))
            y2n = int(min(h_orig, (cy + side/2) * h_orig))
            cropped = frame_rgb[y1n:y2n, x1n:x2n]
            if cropped.size > 0 and min(cropped.shape[:2]) > 20:
                # update template for tracking
                try:
                    tpl = cv2.resize(cropped, (int(min(160, cropped.shape[1])),
int(min(160, cropped.shape[0]))))
                    st.session_state['prev_hand_template'] = tpl.tolist()
                except Exception:
                    pass

                lm = _attach_center(lm, (x1n, y1n, x2n, y2n), frame_rgb.shape[:2])
                return cropped, (x1n, y1n, x2n, y2n), lm, False
    except Exception:
        continue
except Exception as e:
    logger.debug(f"Proposal-based detection failed: {e}")

# No detection found in any variant -> fallback heuristic
if allow_fallback:
    fallback_crop, fallback_bbox = _fallback_hand_crop(frame_rgb)

```



```
    if fallback_crop is not None:
        try:
            tpl = cv2.resize(fallback_crop, (int(min(160, fallback_crop.shape[1])),
int(min(160, fallback_crop.shape[0]))))
            st.session_state['prev_hand_template'] = tpl.tolist()
        except Exception:
            pass
        lm_fb = _attach_center(None, fallback_bbox, frame_rgb.shape[:2])
        return fallback_crop, fallback_bbox, lm_fb, True

return None, None, None, False
```

8. ✓ Hand Rotation Normalization

```
# (Server webcam capture thread removed – realtime tab deprecated)
# HAND ROTATION NORMALIZATION
def normalize_hand_rotation(hand_crop: np.ndarray, landmarks, crop_bbox: tuple = None,
                           original_frame_shape: tuple = None) -> Tuple[np.ndarray, bool,
float]:
    """Normalize hand orientation to upright position using wrist-to-middle-finger axis.
    IMPROVED: Only applies rotation when landmarks are reliable.
    Checks landmark confidence and geometric validity before rotating.
    Uses landmarks 0 (wrist) and 9 (middle finger MCP) to determine hand axis.
    Rotates so the hand points upward (fingers toward top of image).
    Args:
        hand_crop: RGB image of cropped hand
        landmarks: MediaPipe hand landmarks object
        crop_bbox: (x1, y1, x2, y2) bounding box of crop in original image
        original_frame_shape: (height, width) of original frame
    Returns:
        (rotated_crop, rotation_was_applied, angle_applied)
    """
    if hand_crop is None or hand_crop.size == 0:
        return hand_crop, False, 0.0

    if landmarks is None:
        return hand_crop, False, 0.0

    try:
        # Get landmark points
        lm_list = landmarks.landmark if hasattr(landmarks, 'landmark') else landmarks
        if len(lm_list) < 21: # MediaPipe hands has 21 landmarks
            return hand_crop, False, 0.0

        # Wrist (0), Middle finger MCP (9), and Middle finger tip (12)
        wrist = lm_list[0]
        middle_mcp = lm_list[9]
        middle_tip = lm_list[12]

        # Get crop dimensions
        h, w = hand_crop.shape[:2]

        # Map normalized landmark coords to crop pixel coords
        if crop_bbox is not None and original_frame_shape is not None:
            x1, y1, x2, y2 = crop_bbox
            orig_h, orig_w = original_frame_shape[:2]
            # Landmarks are normalized to original frame, map to crop
            wrist_x = (wrist.x * orig_w - x1) if hasattr(wrist, 'x') else 0
            wrist_y = (wrist.y * orig_h - y1) if hasattr(wrist, 'y') else 0
            mcp_x = (middle_mcp.x * orig_w - x1) if hasattr(middle_mcp, 'x') else 0
            mcp_y = (middle_mcp.y * orig_h - y1) if hasattr(middle_mcp, 'y') else 0
            tip_x = (middle_tip.x * orig_w - x1) if hasattr(middle_tip, 'x') else 0
            tip_y = (middle_tip.y * orig_h - y1) if hasattr(middle_tip, 'y') else 0
        else:
            # Assume landmarks normalized to crop
            wrist_x = (wrist.x * w) if hasattr(wrist, 'x') else wrist[0] * w
            wrist_y = (wrist.y * h) if hasattr(wrist, 'y') else wrist[1] * h
            mcp_x = (middle_mcp.x * w) if hasattr(middle_mcp, 'x') else middle_mcp[0] * w
            mcp_y = (middle_mcp.y * h) if hasattr(middle_mcp, 'y') else middle_mcp[1] * h
            tip_x = (middle_tip.x * w) if hasattr(middle_tip, 'x') else middle_tip[0] * w
            tip_y = (middle_tip.y * h) if hasattr(middle_tip, 'y') else middle_tip[1] * h

        # VALIDATION: Check if landmarks are within reasonable bounds
        # Landmarks should be mostly inside the crop
        points = [(wrist_x, wrist_y), (mcp_x, mcp_y), (tip_x, tip_y)]
        margin = 0.15 # Allow 15% outside
        inside_count = sum(1 for px, py in points
                           if -w*margin < px < w*(1+margin) and -h*margin < py < h*
(1+margin))
        if inside_count < 2:
            # Too many landmarks outside crop - unreliable
            return hand_crop, False, 0.0

        # VALIDATION: Check vector length (wrist to MCP)
        dx = mcp_x - wrist_x
        dy = mcp_y - wrist_y
```

```

vector_len = np.sqrt(dx*dx + dy*dy)
# Vector should be at least 10% of crop size to be meaningful
min_vector_len = min(w, h) * 0.10
if vector_len < min_vector_len:
    return hand_crop, False, 0.0

# Calculate angle from wrist to middle MCP
# We want fingers pointing UP (negative y direction in image coords)
# atan2(dx, -dy) gives angle from vertical axis
angle = np.degrees(np.arctan2(dx, -dy))

# VALIDATION: Only apply rotation for significant angles
# Small angles (< 10°) don't need correction
if abs(angle) < 10:
    return hand_crop, False, 0.0

# VALIDATION: Skip extreme rotations (> 60°) as they might be errors
# Most ASL signs don't require >60° rotation
if abs(angle) > 60:
    return hand_crop, False, 0.0

# Rotate the crop to make hand upright
center = (w / 2, h / 2)
M = cv2.getRotationMatrix2D(center, angle, 1.0)
# Rotate with border replication to avoid black corners
rotated = cv2.warpAffine(
    hand_crop, M, (w, h),
    flags=cv2.INTER_LINEAR,
    borderMode=cv2.BORDER_REPLICATE
)
return rotated, True, angle
except Exception as e:
    logger.debug(f"Hand rotation normalization failed: {e}")
    return hand_crop, False, 0.0
except Exception as e:
    logger.debug(f"Hand rotation normalization failed: {e}")
    return hand_crop

```

9. ✓ Multiple Rotations Fallback

```
def try_multiple_rotations(hand_crop: np.ndarray, model_name: str = None) -> Tuple[str,
float, np.ndarray, np.ndarray, int]:
    """Try multiple rotations and return the one with highest confidence.
    This is a fallback when landmarks aren't reliable - try 0°, 90°, 180°, 270°
    and pick the rotation that gives highest model confidence.
    Args:
        hand_crop: RGB image of cropped hand
        model_name: Which model to use (default: best available)
    Returns:
        (prediction, confidence, best_probs, best_rotated_crop, best_angle)
    """
    if hand_crop is None or hand_crop.size == 0:
        return "No Hand", 0.0, None, hand_crop

    # Use EfficientNetV2B3 or first available model for rotation search
    if model_name is None:
        model_name = 'EfficientNetV2B3' if 'EfficientNetV2B3' in MODELS else
list(MODELS.keys())[0]

    model = MODELS.get(model_name)
    if model is None:
        return "Error", 0.0, None, hand_crop

    h, w = hand_crop.shape[:2]
    center = (w / 2, h / 2)
    best_pred = "Unknown"
    best_conf = 0.0
    best_probs = None
    best_crop = hand_crop
    best_angle = 0

    # Try 4 main rotations (0°, 90°, 180°, 270°)
    rotations = [0, 90, 180, 270]
    for angle in rotations:
        try:
            if angle == 0:
                rotated = hand_crop
            else:
                M = cv2.getRotationMatrix2D(center, angle, 1.0)
                rotated = cv2.warpAffine(hand_crop, M, (w, h),
                                        flags=cv2.INTER_LINEAR,
                                        borderMode=cv2.BORDER_REPLICATE)

            # Predict
            input_data = preprocess_for_inference(rotated, model_name)
            output = model.predict(input_data, verbose=0)[0]
            probs = output / (output.sum() + 1e-9)
            top_idx = int(np.argmax(probs))
            conf = float(probs[top_idx])

            if conf > best_conf:
                best_conf = conf
                best_pred = CLASS_NAMES[top_idx]
                best_probs = probs
                best_crop = rotated
                best_angle = angle
        except Exception as e:
            logger.debug(f"Rotation {angle}° failed: {e}")
            continue

    return best_pred, best_conf, best_probs, best_crop, best_angle
```

10. ✓ Preprocessing for Inference

```
# PREPROCESSING
# Import model-specific preprocessing functions to match training
try:
    from tensorflow.keras.applications.efficientnet_v2 import preprocess_input as
effnet_preprocess
    from tensorflow.keras.applications.resnet50 import preprocess_input as
resnet_preprocess
    from tensorflow.keras.applications.inception_v3 import preprocess_input as
inception_preprocess
except ImportError:
    # Fallback if imports fail
    effnet_preprocess = None
    resnet_preprocess = None
    inception_preprocess = None

def preprocess_for_inference(image_rgb: np.ndarray, model_name: str) -> np.ndarray:
    """Preprocess image for model inference - MUST MATCH TRAINING PREPROCESSING!
    CRITICAL: Training code-2 used model-specific preprocess_input for EfficientNetV2B3.
    Training code-1 used simple /255.0 for ResNet50 and InceptionV3.
    We need to match exactly what was used during training for each model.
    """

    # Ensure uint8 input and valid range
    if image_rgb is None:
        raise ValueError("image_rgb is None")
    if image_rgb.dtype != np.uint8:
        try:
            if float(np.max(image_rgb)) <= 1.0:
                image_rgb = (image_rgb * 255.0).astype(np.uint8)
            else:
                image_rgb = image_rgb.astype(np.uint8)
        except Exception:
            image_rgb = image_rgb.astype(np.uint8)

    # Resize to model input size
    img_resized = cv2.resize(image_rgb, (IMG_SIZE, IMG_SIZE))

    # Convert to float32 for preprocessing
    img_float = img_resized.astype(np.float32)

    # Model-specific preprocessing to MATCH TRAINING EXACTLY
    if model_name == 'EfficientNetV2B3' and effnet_preprocess is not None:
        # EfficientNetV2B3 was trained with effnet_preprocess (scales to [-1, 1])
        img_preprocessed = effnet_preprocess(img_float)
    elif model_name == 'ResNet50' and resnet_preprocess is not None:
        # ResNet50 training uses resnet_preprocess (ImageNet mean subtraction, scales to ~
[-1, 1])
        # FIXED: Must use resnet_preprocess, NOT /255.0!
        img_preprocessed = resnet_preprocess(img_float)
    elif model_name == 'InceptionV3' and inception_preprocess is not None:
        # InceptionV3 training uses inception_preprocess (scales to [-1, 1])
        img_preprocessed = inception_preprocess(img_float)
    else:
        # Fallback: simple /255.0 normalization
        img_preprocessed = img_float / 255.0

    return np.expand_dims(img_preprocessed, axis=0)

def _fallback_hand_crop(frame_rgb: np.ndarray, min_size: int = 40) ->
Tuple[Optional[np.ndarray], Optional[tuple]]:
    """Heuristic fallback to produce a hand crop when MediaPipe fails.
    Strategy:
    1. Try simple skin-color segmentation in HSV and take largest contour bbox.
    2. If that fails, return a centered square crop (50% of min dimension).
    Returns (crop_rgb, bbox) or (None, None) on failure.
    """
    try:
        h, w = frame_rgb.shape[:2]
        # 1) Skin color mask in HSV (heuristic)
        hsv = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2HSV)
        # Common-ish skin color range; not perfect but helps as fallback
        lower = np.array([0, 30, 60], dtype=np.uint8)
        upper = np.array([25, 200, 255], dtype=np.uint8)
```

```

mask = cv2.inRange(hsv, lower, upper)
# Morphology to clean mask
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel, iterations=2)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel, iterations=2)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
if contours:
    # choose largest contour
    largest = max(contours, key=cv2.contourArea)
    x, y, cw, ch = cv2.boundingRect(largest)
    area = cw * ch
    if area > (min_size * min_size):
        # Expand bbox slightly for context
        pad = int(0.2 * max(cw, ch))
        x1 = max(0, x - pad)
        y1 = max(0, y - pad)
        x2 = min(w, x + cw + pad)
        y2 = min(h, y + ch + pad)
        crop = frame_rgb[y1:y2, x1:x2]
        if crop.size > 0 and min(crop.shape[:2]) > 20:
            return crop, (x1, y1, x2, y2)
# 2) Center crop as last resort (50% of min dimension)
side = int(0.5 * min(h, w))
if side >= min_size:
    cx = w // 2
    cy = h // 2
    x1 = max(0, cx - side//2)
    y1 = max(0, cy - side//2)
    x2 = min(w, x1 + side)
    y2 = min(h, y1 + side)
    crop = frame_rgb[y1:y2, x1:x2]
    if crop.size > 0 and min(crop.shape[:2]) > 20:
        return crop, (x1, y1, x2, y2)
except Exception as e:
    logger.debug(f"Fallback crop failed: {e}")

return None, None

```


11. ✓ Single Model Prediction

```
# SINGLE MODEL PREDICTION (CPU-OPTIMIZED FOR REAL-TIME)
# Default model for real-time inference (best balance of speed/accuracy on CPU)
REALTIME_MODEL = 'InceptionV3' # InceptionV3 often most reliable

def predict_with_model(hand_crop_rgb: np.ndarray, model, model_name: str) -> Tuple[str,
float, Optional[np.ndarray]]:
    """Direct prediction with a specific model - thread-safe version.
    Args:
        hand_crop_rgb: RGB image of cropped hand
        model: The loaded Keras model
        model_name: Name for preprocessing selection
    Returns:
        (class_name, confidence, probabilities)
    """
    try:
        if hand_crop_rgb is None or hand_crop_rgb.size == 0:
            return "No Hand", 0.0, None

        if model is None:
            return "Error", 0.0, None

        # Preprocess and predict
        input_data = preprocess_for_inference(hand_crop_rgb, model_name)
        output = model.predict(input_data, verbose=0)[0]

        # Normalize probabilities
        probs = output / (output.sum() + 1e-9)
        top_idx = int(np.argmax(probs))
        return CLASS_NAMES[top_idx], float(probs[top_idx]), probs
    except Exception as e:
        logger.error(f"Predict error: {e}")
        return "Error", 0.0, None

def single_model_predict(hand_crop_rgb: np.ndarray,
                        model_name: str = None) -> Tuple[str, float,
Optional[np.ndarray]]:
    """Single model prediction optimized for real-time CPU inference.
    Uses only one model for fast inference. Ideal for Video/Live Camera tabs.
    Args:
        hand_crop_rgb: RGB image of cropped hand
        model_name: Which model to use (default: user selection or REALTIME_MODEL)
    Returns:
        (class_name, confidence, probabilities)
    """
    try:
        if hand_crop_rgb is None or hand_crop_rgb.size == 0:
            return "No Hand", 0.0, None

        # Use user-selected model from sidebar, or default REALTIME_MODEL
        if model_name is None:
            try:
                model_name = st.session_state.get('realtime_model', REALTIME_MODEL)
            except Exception:
                model_name = REALTIME_MODEL

        model = MODELS.get(model_name)
        if model is None:
            # Fallback to any available model
            if len(MODELS) > 0:
                model_name = list(MODELS.keys())[0]
                model = MODELS[model_name]
            else:
                return "Error", 0.0, None

        return predict_with_model(hand_crop_rgb, model, model_name)
    except Exception as e:
        logger.error(f"Single model predict error: {e}")
        return "Error", 0.0, None
```


12. ✓ Ensemble Prediction

```
# ENSEMBLE PREDICTION (FOR IMAGE TAB - ACCURACY PRIORITY)
def ensemble_predict(hand_crop_rgb: np.ndarray,
                    threshold: float = 0.3) -> Tuple[str, float, Optional[np.ndarray],
Dict]:
    """Run ensemble prediction - use for Image tab where accuracy is priority"""
    try:
        if hand_crop_rgb is None or hand_crop_rgb.size == 0:
            return "No Hand", 0.0, None, {}

        all_probs = []
        model_votes = {}

        # Respect enabled models and optional weights from sidebar/session
        enabled = st.session_state.get('enabled_models', list(MODELS.keys()))
        model_weights = st.session_state.get('model_weights', None)

        for model_name, model in MODELS.items():
            if model_name not in enabled:
                continue

            try:
                input_data = preprocess_for_inference(hand_crop_rgb, model_name)
                output = model.predict(input_data, verbose=0)[0]
                # Normalize
                probs = output / output.sum()
                all_probs.append(probs)
                top_pred = np.argmax(probs)
                model_votes[model_name] = {
                    'class': CLASS_NAMES[top_pred],
                    'confidence': float(probs[top_pred])
                }
            except Exception as e:
                logger.error(f"{model_name} failed: {e}")

        if len(all_probs) == 0:
            return "Error", 0.0, None, {}

        # Compute weighted or simple average
        if model_weights:
            # model_weights expected to be dict {name: weight}
            total_w = 0.0
            weighted = None
            for name, probs in zip([m for m in MODELS.keys() if m in enabled], all_probs):
                w = float(model_weights.get(name, 1.0))
                total_w += w
                if weighted is None:
                    weighted = probs * w
                else:
                    weighted = weighted + probs * w

            if weighted is None:
                avg_probs = np.mean(all_probs, axis=0)
            else:
                avg_probs = weighted / max(1e-6, total_w)
        else:
            avg_probs = np.mean(all_probs, axis=0)

        top_idx = np.argmax(avg_probs)
        return CLASS_NAMES[top_idx], float(avg_probs[top_idx]), avg_probs, model_votes
    except Exception as e:
        logger.error(f"Ensemble error: {e}")
        return "Error", 0.0, None, {}
```

13. ✓ Ensemble Prediction with TTA

```
def ensemble_predict_tta(hand_crop_rgb: np.ndarray, tta: bool = False) -> Tuple[str, float,
Optional[np.ndarray], Dict]:
    """Ensemble prediction with optional simple TTA (scaled crops + horizontal flip).
    Returns same tuple as `ensemble_predict` (label, confidence, avg_probs, model_votes)
    where avg_probs is averaged across models and TTA crops.
    """
    try:
        if hand_crop_rgb is None or hand_crop_rgb.size == 0:
            return "No Hand", 0.0, None, {}

        # Build crop variants (expanded TTA set)
        crops = [hand_crop_rgb]
        if tta:
            h, w = hand_crop_rgb.shape[:2]
            scales = [0.9, 1.0, 1.1]
            rotations = [-8, 0, 8]
            for s in scales:
                try:
                    scaled = cv2.resize(hand_crop_rgb, (int(w * s), int(h * s)))
                    ch, cw = scaled.shape[:2]
                    # center-crop or pad to original size
                    if ch >= h and cw >= w:
                        sx = (cw - w) // 2
                        sy = (ch - h) // 2
                        crop0 = scaled[sy:sy + h, sx:sx + w]
                    else:
                        # pad
                        pad_h = max(0, h - ch)
                        pad_w = max(0, w - cw)
                        crop0 = cv2.copyMakeBorder(scaled, pad_h//2, pad_h - pad_h//2,
pad_w//2, pad_w - pad_w//2, cv2.BORDER_CONSTANT, value=[0,0,0])
                        crop0 = cv2.resize(crop0, (w, h))
                    crops.append(crop0)
                except Exception:
                    continue

            # small rotations and flips
            for r in rotations:
                try:
                    M = cv2.getRotationMatrix2D((w/2, h/2), r, 1.0)
                    rot = cv2.warpAffine(hand_crop_rgb, M, (w, h),
borderMode=cv2.BORDER_REPLICATE)
                    crops.append(rot)
                except Exception:
                    pass

            try:
                flip = cv2.flip(hand_crop_rgb, 1)
                crops.append(flip)
            except Exception:
                pass

        # Respect enabled models and optional per-model weights from sidebar/session
        enabled = st.session_state.get('enabled_models', list(MODELS.keys()))
        default_weights = st.session_state.get('model_weights', DEFAULT_ENSEMBLE_WEIGHTS)

        # accumulate probs across crops and models
        accumulated = None
        model_votes = {}
        weight_sum = 0.0

        for model_name, model in MODELS.items():
            if model_name not in enabled:
                continue

            model_probs = []
            for c in crops:
                try:
                    inp = preprocess_for_inference(c, model_name)
                    out = model.predict(inp, verbose=0)[0]
                    probs = out / out.sum()
                    model_probs.append(probs)
```

```

        except Exception as e:
            logger.debug(f"TTA predict failed for {model_name}: {e}")

    if len(model_probs) == 0:
        continue

    # average across crops for this model
    model_avg = np.mean(model_probs, axis=0)
    model_votes[model_name] = {
        'class': CLASS_NAMES[int(np.argmax(model_avg))],
        'confidence': float(np.max(model_avg))
    }

    w = float(default_weights.get(model_name, 1.0))
    weight_sum += w

    if accumulated is None:
        accumulated = model_avg * w
    else:
        accumulated = accumulated + model_avg * w

    if accumulated is None:
        return "Error", 0.0, None, {}

    # weighted average across models
    if weight_sum > 0:
        avg_probs = accumulated / weight_sum
    else:
        avg_probs = accumulated / max(1, len(model_votes))

    # Apply temperature scaling if enabled (calibrates confidence)
    temperature = st.session_state.get('temperature_scaling', DEFAULT_TEMPERATURE)
    if temperature != 1.0:
        avg_probs = apply_temperature_scaling(avg_probs, temperature)

    top_idx = int(np.argmax(avg_probs))
    return CLASS_NAMES[top_idx], float(avg_probs[top_idx]), avg_probs, model_votes
except Exception as e:
    logger.error(f"Ensemble TTA error: {e}")
    return "Error", 0.0, None, {}

```

14. ✓ Temperature Scaling for Confidence Calibration

```

def apply_temperature_scaling(probs: np.ndarray, temperature: float = 1.5) -> np.ndarray:
    """Apply temperature scaling to calibrate prediction confidence.
    Temperature scaling divides logits by T before softmax.
    Higher T = softer, more calibrated probabilities.
    Args:
        probs: Probability distribution (already softmaxed)
        temperature: Scaling factor (1.0 = no change, >1 = softer, <1 = sharper)
    Returns:
        Recalibrated probability distribution
    """
    if temperature == 1.0:
        return probs

    try:
        # Convert probs back to logits (inverse softmax)
        # Add small epsilon to avoid log(0)
        eps = 1e-10
        probs_safe = np.clip(probs, eps, 1.0)
        logits = np.log(probs_safe)

        # Apply temperature scaling
        scaled_logits = logits / temperature

        # Re-apply softmax
        exp_logits = np.exp(scaled_logits - np.max(scaled_logits)) # numerical stability
        scaled_probs = exp_logits / exp_logits.sum()

        return scaled_probs
    except Exception:
        return probs

```

15. ✓ Confusion Pair Analysis

```
def handle_confusion_pairs(prediction: str, confidence: float, probs: np.ndarray,
                           model_votes: Dict, confusion_threshold: float = 0.6) ->
Tuple[str, float, str]:
    """Handle commonly confused letter pairs with additional analysis.
    When confidence is low and prediction is in a known confusion pair,
    analyze model agreement and probability distribution to potentially
    adjust or flag the prediction.
    Args:
        prediction: Current predicted letter
        confidence: Prediction confidence
        probs: Full probability distribution
        model_votes: Individual model predictions
        confusion_threshold: Below this, apply confusion analysis
    Returns:
        (final_prediction, adjusted_confidence, analysis_note)
    """
    analysis_note = ""

    # Only apply for low-confidence predictions
    if confidence >= confusion_threshold:
        return prediction, confidence, ""

    # Check if prediction is in a confusion pair
    if prediction not in CONFUSION_PAIRS:
        return prediction, confidence, ""

    confused_with = CONFUSION_PAIRS[prediction]

    # Get top-2 predictions from probability distribution
    sorted_indices = np.argsort(probs)[::-1]
    top1_idx = sorted_indices[0]
    top2_idx = sorted_indices[1]
    top1_class = CLASS_NAMES[top1_idx]
    top2_class = CLASS_NAMES[top2_idx]
    top1_prob = probs[top1_idx]
    top2_prob = probs[top2_idx]

    # Check if top-2 are in a confusion pair
    if top2_class in confused_with:
        prob_gap = top1_prob - top2_prob
        # If gap is very small, both are equally likely
        if prob_gap < 0.1:
            analysis_note = f"Uncertain: Could be {top1_class} or {top2_class} (similar confidence)"
        elif prob_gap < 0.2:
            analysis_note = f"Likely {top1_class}, but {top2_class} is also possible"

    # Check model agreement
    if model_votes:
        votes = [v['class'] for v in model_votes.values()]
        vote_counts = Counter(votes)
        # If models disagree significantly
        if len(vote_counts) > 1:
            most_common = vote_counts.most_common()
            if len(most_common) >= 2:
                top_vote, top_count = most_common[0]
                second_vote, second_count = most_common[1]
                # If models are split on confused pairs
                if second_vote in CONFUSION_PAIRS.get(top_vote, []):
                    if top_count == second_count:
                        analysis_note = f"Models split: {top_count}× {top_vote} vs {second_count}× {second_vote}"
                    elif second_count >= 1:
                        if not analysis_note:
                            analysis_note = f"Some models predicted {second_vote} instead"

    return prediction, confidence, analysis_note
```

16. ✓ Visualization Functions

```
# VISUALIZATION
def draw_hand_overlay(image_rgb: np.ndarray,
                      bbox: tuple,
                      landmarks: any,
                      prediction: str = None,
                      confidence: float = None,
                      hand_crop: Optional[np.ndarray] = None,
                      enable_xai: bool = False,
                      gradcam_map: Optional[np.ndarray] = None) -> np.ndarray:
    """Draw detection overlay with proper Grad-CAM visualization."""
    annotated = image_rgb.copy()

    # GRAD-CAM OVERLAY - Apply to the ORIGINAL full image

    if enable_xai and gradcam_map is not None and bbox is not None:
        try:
            x1, y1, x2, y2 = bbox
            h_bbox = y2 - y1
            w_bbox = x2 - x1

            if h_bbox > 0 and w_bbox > 0:
                # Resize Grad-CAM heatmap to match bbox size
                heatmap_resized = cv2.resize(gradcam_map, (w_bbox, h_bbox))

                # Convert to uint8 and apply colormap
                heatmap_uint8 = (heatmap_resized * 255).astype(np.uint8)
                heatmap_colored = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
                heatmap_rgb = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)

                # Extract the original region from the input image (NOT annotated)
                original_region = image_rgb[y1:y2, x1:x2].copy()

                # Blend: make heatmap stronger so it's clearly visible
                blended = cv2.addWeighted(original_region, 0.4, heatmap_rgb, 0.6, 0)

                # Place blended region back into annotated image
                annotated[y1:y2, x1:x2] = blended
            except Exception as e:
                logger.debug(f"Grad-CAM overlay failed: {e}")

        # LAPLACIAN SALIENCY (if enabled and Grad-CAM not used)

    elif enable_xai and hand_crop is not None and gradcam_map is None:
        try:
            # Simple edge-based saliency as fallback
            gray = cv2.cvtColor(hand_crop, cv2.COLOR_RGB2GRAY)
            lap = cv2.Laplacian(gray, cv2.CV_64F)
            lap = np.absolute(lap)
            if lap.max() > 0:
                lap = (lap / lap.max() * 255).astype(np.uint8)
            else:
                lap = (lap * 0).astype(np.uint8)

            if bbox:
                x1, y1, x2, y2 = bbox
                h = max(1, y2 - y1)
                w = max(1, x2 - x1)
                heat = cv2.resize(lap, (w, h))
                heat_color = cv2.applyColorMap(heat, cv2.COLORMAP_JET)
                heat_rgb = cv2.cvtColor(heat_color, cv2.COLOR_BGR2RGB)

                original_region = image_rgb[y1:y2, x1:x2].copy()
                blended = cv2.addWeighted(original_region, 0.7, heat_rgb, 0.3, 0)
                annotated[y1:y2, x1:x2] = blended
            except Exception as e:
                logger.debug(f"Laplacian overlay failed: {e}")

    # DRAW LANDMARKS
```

```

if landmarks:
    mp_drawing = mp.solutions.drawing_utils
    mp_hands = mp.solutions.hands

    try:
        mp_drawing.draw_landmarks(
            annotated, landmarks, mp_hands.HAND_CONNECTIONS,
            mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2, circle_radius=3),
            mp_drawing.DrawingSpec(color=(255, 0, 0), thickness=2)
        )
    except:
        # Fallback: draw simple points
        try:
            h_img, w_img = annotated.shape[:2]
            for lm in getattr(landmarks, 'landmark', []):
                try:
                    px = int(lm.x * w_img)
                    py = int(lm.y * h_img)
                    cv2.circle(annotated, (px, py), 6, (0, 255, 255), -1)
                except Exception:
                    continue
        except Exception:
            pass

# DRAW BOUNDING BOX AND TEXT

if bbox:
    x1, y1, x2, y2 = bbox
    cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 3)

    # Add prediction text
    if prediction and confidence:
        text = f"{prediction}: {confidence:.0%}"
        # Background for text
        (text_w, text_h), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.8, 2)
        cv2.rectangle(annotated, (x1, y1-text_h-15), (x1+text_w+10, y1), (0, 255, 0),
-1)

        cv2.putText(annotated, text, (x1+5, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 0), 2)

return annotated

```


17. ✓ Full Rotated Overlay Function

```
def draw_full_rotated_overlay(
    image_rgb: np.ndarray,
    bbox: Optional[tuple],
    landmarks: any = None,
    prediction: Optional[str] = None,
    confidence: Optional[float] = None,
    enable_xai: bool = False,
    gradcam_map: Optional[np.ndarray] = None,
    rotation_applied: bool = False,
    rotation_angle: float = 0.0,
) -> np.ndarray:
    """
    Draw Grad-CAM over the ENTIRE image, and rotate the whole image if needed.
    Steps:
    - Start from the original full image.
    - Place the Grad-CAM heatmap into the hand bbox region.
    - (Optional) Rotate both image and heatmap together by the same angle.
    - Blend heatmap over the entire rotated image.
    - Draw a bounding box and prediction text on the rotated result.
    """
    if image_rgb is None or image_rgb.size == 0:
        return image_rgb

    visual = image_rgb.copy()
    h_full, w_full = visual.shape[:2]

    # Draw landmarks and bbox on the ORIGINAL frame first
    # (they will be rotated together with the image)

    if landmarks:
        mp_drawing = mp.solutions.drawing_utils
        mp_hands = mp.solutions.hands

        try:
            mp_drawing.draw_landmarks(
                visual,
                landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2, circle_radius=3),
                mp_drawing.DrawingSpec(color=(255, 0, 0), thickness=2),
            )
        except Exception:
            # Fallback: simple landmark points
            try:
                for lm in getattr(landmarks, "landmark", []):
                    try:
                        px = int(lm.x * w_full)
                        py = int(lm.y * h_full)
                        cv2.circle(visual, (px, py), 4, (0, 255, 255), -1)
                    except Exception:
                        continue
            except Exception:
                pass

    if bbox is not None:
        try:
            x1, y1, x2, y2 = bbox
            cv2.rectangle(visual, (x1, y1), (x2, y2), (0, 255, 0), 3)
        except Exception:
            pass

    # Rotate whole image (and heatmap) if rotation was applied

    new_bbox = bbox
    if rotation_applied and abs(rotation_angle) > 1.0:
        try:
            center = (w_full / 2.0, h_full / 2.0)
            M = cv2.getRotationMatrix2D(center, rotation_angle, 1.0)
            visual = cv2.warpAffine(
                visual,
```



```

        M,
        (w_full, h_full),
        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_REPLICATE,
    )

    # Transform bbox corners into rotated coordinates
    if bbox is not None:
        x1, y1, x2, y2 = bbox
        pts = np.array(
            [[x1, y1], [x2, y1], [x2, y2], [x1, y2]], dtype=np.float32
        )
        ones = np.ones((pts.shape[0], 1), dtype=np.float32)
        pts_h = np.hstack([pts, ones]) # (4,3)
        M_full = M # 2x3
        pts_rot = pts_h @ M_full.T
        xs = pts_rot[:, 0]
        ys = pts_rot[:, 1]
        nx1, ny1, nx2, ny2 = (
            int(xs.min()),
            int(ys.min()),
            int(xs.max()),
            int(ys.max()),
        )
        # Clamp to image bounds
        nx1 = max(0, min(w_full - 1, nx1))
        ny1 = max(0, min(h_full - 1, ny1))
        nx2 = max(0, min(w_full - 1, nx2))
        ny2 = max(0, min(h_full - 1, ny2))
        new_bbox = (nx1, ny1, nx2, ny2)
    except Exception as e:
        logger.debug(f"Rotating full image for overlay failed: {e}")

    # Build and blend FULL-SIZE heatmap over the (possibly rotated) image
    # (when enabled). Use a softer blend so it matches the background.
    # Here we build the heatmap in the *rotated* coordinates, inside new_bbox.
    # If Grad-CAM is weak/unavailable, fall back to landmark-based heatmap.

    if enable_xai and new_bbox is not None:
        try:
            x1, y1, x2, y2 = new_bbox
            h_box = max(1, y2 - y1)
            w_box = max(1, x2 - x1)

            # ALWAYS try to use real Grad-CAM first (real XAI)
            use_gradcam = False
            hm = None

            if gradcam_map is not None:
                # Resize Grad-CAM to bbox size (it's from the crop, so resize to match
bbox)
                hm_box = cv2.resize(gradcam_map, (w_box, h_box),
interpolation=cv2.INTER_CUBIC)
                # Check if Grad-CAM has meaningful activations
                vals = hm_box.flatten()
                if len(vals) > 0:
                    val_range = vals.max() - vals.min()
                    val_std = vals.std()
                    val_max = vals.max()
                    # More lenient check - if we have any meaningful signal, use it
                    if val_max > 0.005 and val_std > 0.001:
                        use_gradcam = True

            if use_gradcam:
                # Use REAL Grad-CAM - this is the actual model attention!
                hm = np.zeros((h_full, w_full), dtype=np.float32)
                # Place heatmap directly in bbox (no expansion) for focused coverage on
hand
                hm[y1:y2, x1:x2] = hm_box

                # Apply MODERATE Gaussian smoothing - focused on hand area, not too spread
                # Smaller kernel = less area coverage, more focused on actual hand
                kernel_size = max(7, min(h_box, w_box) // 12) # Reduced for less spread
                if kernel_size % 2 == 0:

```

```

        kernel_size += 1

# Moderate sigma for focused coverage
sigma = kernel_size / 2.5 # Reduced for less spread

hm = cv2.GaussianBlur(hm, (kernel_size, kernel_size), sigmaX=sigma,
sigmaY=sigma)

# Normalize to ensure good color range (but preserve relative intensities)
mask = hm > 0
if np.any(mask):
    vals = hm[mask]
    # Use optimized percentiles to make colors MORE visible
    lo = np.percentile(vals, 10.0) # Lower threshold - more aggressive
    hi = np.percentile(vals, 95.0) # Upper threshold - highlight top

    if hi > lo:
        hm = (hm - lo) / (hi - lo)
        hm = np.clip(hm, 0.0, 1.0)
    else:
        # Fallback: simple max normalization
        if hm.max() > 0:
            hm = hm / hm.max()
else:
    use_gradcam = False

# ONLY use landmark fallback if Grad-CAM completely failed (last resort)
# Note: This is NOT real XAI - it's just a visual approximation
if not use_gradcam and landmarks is not None:
    logger.debug("Real Grad-CAM unavailable, using landmark-based fallback (not
real XAI)")

# Create heatmap from landmarks BEFORE rotation
# (landmarks are in original image coords)
hm_orig = create_landmark_based_heatmap(
    (h_full, w_full),
    landmarks,
    bbox if not rotation_applied else None
)
if hm_orig is not None:
    # If rotation was applied, rotate the heatmap too
    if rotation_applied and abs(rotation_angle) > 1.0:
        center = (w_full / 2.0, h_full / 2.0)
        M = cv2.getRotationMatrix2D(center, rotation_angle, 1.0)
        hm = cv2.warpAffine(
            hm_orig,
            M,
            (w_full, h_full),
            flags=cv2.INTER_LINEAR,
            borderMode=cv2.BORDER_CONSTANT,
            borderValue=0
        )
    else:
        hm = hm_orig

# Apply heatmap if we have one
if hm is not None and hm.max() > 0:
    # Normalize to [0,1] if not already
    if hm.max() > 1.0 or hm.min() < 0.0:
        hm = (hm - hm.min()) / (hm.max() - hm.min() + 1e-9)

# Apply power curve to enhance hotspots (makes red/yellow MORE visible)
# Lower exponent = more aggressive enhancement of hotspots
hm = np.power(hm, 0.65) # More aggressive for better color visibility

# Boost saturation to make colors more vibrant and visible
heatmap_uint8 = (hm * 255).astype(np.uint8)
heatmap_colored = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
heatmap_rgb = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)

# Enhance color saturation and brightness for BETTER visibility
hsv = cv2.cvtColor(heatmap_rgb, cv2.COLOR_RGB2HSV).astype(np.float32)
hsv[:, :, 1] = np.clip(hsv[:, :, 1] * 1.4, 0, 255) # Increase saturation
hsv[:, :, 2] = np.clip(hsv[:, :, 2] * 1.1, 0, 255) # Increase brightness

```

```

        heatmap_rgb = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2RGB)

        # Blend: Optimized mix for better visibility while keeping image readable
        if use_gradcam:
            # Real Grad-CAM: balanced blend for clear visibility
            visual = cv2.addWeighted(visual, 0.60, heatmap_rgb, 0.40, 0)
        else:
            # Fallback: slightly softer blend
            visual = cv2.addWeighted(visual, 0.70, heatmap_rgb, 0.30, 0)
    except Exception as e:
        logger.debug(f"Full-image heatmap blend failed: {e}")

# Draw prediction text on final rotated image
if prediction and confidence is not None:
    try:
        text = f"{prediction}: {confidence:.0%}"
        (text_w, text_h), _ = cv2.getTextSize(
            text, cv2.FONT_HERSHEY_SIMPLEX, 0.8, 2
        )
        x_text, y_text = 10, 30
        cv2.rectangle(
            visual,
            (x_text, y_text - text_h - 10),
            (x_text + text_w + 10, y_text + 5),
            (0, 255, 0),
            -1,
        )
        cv2.putText(
            visual,
            text,
            (x_text + 5, y_text),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (0, 0, 0),
            2,
        )
    except Exception as e:
        logger.debug(f"Prediction text on full rotated overlay failed: {e}")

return visual

```

18. ✓ Hand Overlay on Rotated Crop

```
def draw_hand_overlay_on_rotated_crop(
    hand_crop: np.ndarray,
    prediction: str = None,
    confidence: float = None,
    enable_xai: bool = False,
    gradcam_map: Optional[np.ndarray] = None,
) -> np.ndarray:
    """
    Draw overlays directly on the (possibly rotated) hand crop.
    This is used for the image-upload pipeline so that:
    - The displayed image is the same rotated crop we fed to the model
    - The Grad-CAM heatmap is applied over the FULL rotated crop,
      not just the small bounding-box region on the original frame.
    """
    if hand_crop is None or hand_crop.size == 0:
        return hand_crop

    annotated = hand_crop.copy()
    h, w = annotated.shape[:2]

    # GRAD-CAM OVERLAY ON FULL ROTATED CROP

    if enable_xai and gradcam_map is not None:
        try:
            # Resize Grad-CAM map to the full crop size
            heatmap_resized = cv2.resize(gradcam_map, (w, h))
            heatmap_uint8 = (heatmap_resized * 255).astype(np.uint8)
            heatmap_colored = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
            heatmap_rgb = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)

            # Blend over the entire crop
            annotated = cv2.addWeighted(annotated, 0.6, heatmap_rgb, 0.4, 0)
        except Exception as e:
            logger.debug(f"Grad-CAM overlay on rotated crop failed: {e}")

    # DRAW SIMPLE BORDER (OPTIONAL)

    try:
        cv2.rectangle(annotated, (0, 0), (w - 1, h - 1), (0, 255, 0), 2)
    except Exception:
        pass

    # DRAW PREDICTION TEXT

    if prediction and confidence is not None:
        try:
            text = f"{prediction}: {confidence:.0%}"
            (text_w, text_h), _ = cv2.getTextSize(
                text, cv2.FONT_HERSHEY_SIMPLEX, 0.8, 2
            )
            # Text background at the top of the crop
            cv2.rectangle(
                annotated,
                (5, 5),
                (5 + text_w + 10, 5 + text_h + 10),
                (0, 255, 0),
                -1,
            )
            cv2.putText(
                annotated,
                text,
                (10, 5 + text_h + 2),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0, 0, 0),
                2,
            )
        except Exception as e:
            logger.debug(f"Drawing prediction text on rotated crop failed: {e}")
```

return annotated

19. ✓ Synthetic Heatmap from Landmarks

```
# Synthetic heatmap from MediaPipe landmarks (fallback when Grad-CAM is weak)

def create_landmark_based_heatmap(
    image_shape: tuple,
    landmarks: any,
    bbox: Optional[tuple] = None,
) -> Optional[np.ndarray]:
    """
    Create a fallback heatmap from MediaPipe hand landmarks.
    ONLY used when real Grad-CAM completely fails - this is NOT real XAI.
    Creates smooth coverage over the entire hand area, not just points.
    """
    if landmarks is None:
        return None

    try:
        h, w = image_shape[:2]
        heatmap = np.zeros((h, w), dtype=np.float32)

        # Get landmark points
        lm_list = landmarks.landmark if hasattr(landmarks, 'landmark') else []
        if len(lm_list) < 21:
            return None

        # Get ALL landmarks to create full hand coverage
        all_points = []
        for idx, lm in enumerate(lm_list):
            px = int(lm.x * w)
            py = int(lm.y * h)
            # Fingertips get highest weight, then joints, then other points
            if idx in [4, 8, 12, 16, 20]: # Fingertips
                weight = 1.0
            elif idx in [3, 7, 11, 15, 19, 2, 6, 10, 14, 18]: # Key joints
                weight = 0.8
            else:
                weight = 0.5
            all_points.append((px, py, weight))

        if not all_points:
            return None

        # Create MODERATE Gaussian blobs - focused on hand, not too spread
        base_sigma = max(20, min(h, w) // 18) # Reduced for less coverage, more focused

        for px, py, weight in all_points:
            # Create a 2D Gaussian kernel with MODERATE spread
            y_coords, x_coords = np.ogrid[:h, :w]
            sigma = base_sigma * (1.0 + weight * 0.4) # Reduced multiplier for less spread
            gaussian = np.exp(-((x_coords - px)**2 + (y_coords - py)**2) / (2 * sigma**2))
            heatmap += gaussian * weight

        # Normalize to [0, 1]
        if heatmap.max() > 0:
            heatmap = heatmap / heatmap.max()

        # Apply MODERATE smoothing - focused on hand area
        kernel_size = max(15, min(h, w) // 20) # Reduced for less spread
        if kernel_size % 2 == 0:
            kernel_size += 1

        # Moderate sigma for focused coverage
        sigma = kernel_size / 2.5 # Reduced for less spread

        heatmap = cv2.GaussianBlur(heatmap, (kernel_size, kernel_size), sigmaX=sigma,
                                   sigmaY=sigma)

        # Re-normalize after blur
        if heatmap.max() > 0:
            heatmap = heatmap / heatmap.max()

        # Apply power curve to enhance hotspots for better color visibility
```



```
heatmap = np.power(heatmap, 0.7) # More aggressive enhancement

return heatmap
except Exception as e:
    logger.debug(f"Creating landmark-based heatmap failed: {e}")
return None
```


20. ✓ Grad-CAM Utilities

```
# Grad-CAM utilities

def find_last_conv_layer(model: tf.keras.Model) -> Optional[str]:
    for layer in reversed(model.layers):
        from tensorflow.keras import layers as _layers
        if isinstance(layer, (_layers.Conv2D, _layers.SeparableConv2D,
                               _layers.DepthwiseConv2D)):
            return layer.name
        if 'conv' in layer.name.lower():
            return layer.name
    return None

def make_gradcam_heatmap(preprocessed_input: np.ndarray, model: tf.keras.Model) ->
Optional[np.ndarray]:
    """
    Return a REAL Grad-CAM heatmap (2D numpy array, values in [0,1]) for the top predicted
    class.
    This shows where the model is actually focusing - real XAI, not synthetic.
    """
    try:
        last_conv = find_last_conv_layer(model)
        if last_conv is None:
            logger.debug("No convolutional layer found for Grad-CAM")
            return None

        # Build gradient model: input -> [last_conv_output, predictions]
        grad_model = tf.keras.Model([model.inputs], [model.get_layer(last_conv).output,
model.output])

        img_tensor = tf.convert_to_tensor(preprocessed_input)

        # Compute gradients with respect to the predicted class
        with tf.GradientTape(watch_accessed_variables=False) as tape:
            tape.watch(img_tensor)
            conv_outputs, predictions = grad_model(img_tensor)
            class_idx = tf.argmax(predictions[0])
            loss = predictions[:, class_idx]

        # Get gradients of loss w.r.t. conv layer outputs
        grads = tape.gradient(loss, conv_outputs)

        # Global average pooling of gradients (weight each channel)
        pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

        # Weight the conv outputs by the pooled gradients
        conv_outputs = conv_outputs[0] # Remove batch dimension
        heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)

        # ReLU: only show positive contributions
        heatmap = tf.maximum(heatmap, 0)

        # Normalize to [0, 1] - but use robust normalization to avoid all-blue
        heatmap_np = heatmap.numpy()
        heatmap_max = heatmap_np.max()
        if heatmap_max > 1e-9:
            # Use 99th percentile for normalization to highlight top activations
            # This prevents "all blue" by scaling so top 1% activations reach 1.0
            percentile_99 = np.percentile(heatmap_np.flatten(), 99.0)
            if percentile_99 > 1e-9:
                heatmap_np = heatmap_np / percentile_99
                heatmap_np = np.clip(heatmap_np, 0.0, 1.0)
            else:
                heatmap_np = heatmap_np / heatmap_max
        else:
            # All zeros - return None to trigger fallback
            return None

        # Additional check: ensure we have meaningful activations
        if heatmap_np.max() < 0.01 or heatmap_np.std() < 0.001:
            logger.debug(f"Grad-CAM too weak: max={heatmap_np.max():.6f}, std=
```

```
{heatmap_np.std():.6f}")
    return None

    return heatmap_np
except Exception as e:
    logger.debug(f"Grad-CAM computation failed: {e}")
    import traceback
    logger.debug(traceback.format_exc())
    return None
```

21. ✓ Tab 1: Image Upload

```
# TAB 1: IMAGE UPLOAD (ENHANCED)
def tab_image_upload():
    st.header("Image Upload")
    st.markdown("Upload an image containing an ASL sign")

    # Quick fetcher: allow pasting an image URL
    url = st.text_input("Image URL (http/https)", key='image_url')
    if st.button("🖼️ Fetch URL", key='fetch_url'):
        if url:
            try:
                resp = urllib.request.urlopen(url, timeout=10)
                data = resp.read()
                arr = np.asarray(bytearray(data), dtype=np.uint8)
                bgr = cv2.imdecode(arr, cv2.IMREAD_COLOR)
                if bgr is None:
                    st.error("Failed to decode image from URL")
                else:
                    fetched_rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
                    st.session_state['_fetched_image'] = True
                    st.session_state['_fetched_image_rgb'] = fetched_rgb.tolist()
                    st.success("Fetched image successfully")
            except Exception as e:
                st.error(f"Failed to fetch image: {e}")

    uploaded_file = st.file_uploader(
        "Choose an image...",
        type=['jpg', 'jpeg', 'png'],
        key="image_uploader"
    )

    # Support either uploaded file or fetched image
    has_fetched = st.session_state.get('_fetched_image', False)
    if uploaded_file is not None or has_fetched:
        # Read image
        if has_fetched and uploaded_file is None:
            try:
                image_rgb = np.array(st.session_state.get('_fetched_image_rgb'),
dtype=np.uint8)
            except Exception:
                st.error("Failed to load fetched image from memory")
                return
        else:
            file_bytes = np.asarray(bytearray(uploaded_file.read()), dtype=np.uint8)
            image_bgr = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)
            image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

        col1, col2 = st.columns([1, 1])
        with col1:
            st.image(image_rgb, caption="Original Image", use_column_width=True)

    if st.button("Analyze Hand Sign", type="primary", use_container_width=True):
        with st.spinner("Processing..."):
            # Create detector
            detector = mp.solutions.hands.Hands(
                static_image_mode=True,
                max_num_hands=1,
                min_detection_confidence=0.6,
                model_complexity=1
            )

            # Detect hand
            hand_crop, bbox, landmarks, used_fallback = detect_hand_in_frame(
                image_rgb, detector, enhance=True
            )
            detector.close()

            if hand_crop is not None:
                # Read settings
                conf_threshold = st.session_state.get('img_conf_threshold', 0.3)
                enable_xai = st.session_state.get('enable_xai', False)
                use_tta = st.session_state.get('enable_tta', True)
```

```

# STEP 1: ROTATION NORMALIZATION

hand_crop_normalized = hand_crop
rotation_applied = False
rotation_angle = 0.0

if landmarks is not None and not used_fallback:
    hand_crop_normalized, rotation_applied, rotation_angle =
normalize_hand_rotation(
    hand_crop, landmarks, bbox, image_rgb.shape
)

# Do prediction on normalized crop
prediction, confidence, probs, model_votes =
ensemble_predict_tta(hand_crop_normalized, tta=use_tta)

# Try multiple rotations if confidence is low
best_rotation_crop = hand_crop_normalized
if confidence < 0.5:
    rot_pred, rot_conf, rot_probs, rot_crop, rot_angle =
try_multiple_rotations(hand_crop)
    if rot_conf > confidence:
        prediction, confidence, probs, model_votes =
ensemble_predict_tta(rot_crop, tta=use_tta)
        best_rotation_crop = rot_crop
        rotation_applied = True
        rotation_angle = float(rot_angle)
        st.info(f"Applied rotation correction: {rot_angle}°")
elif rotation_applied and abs(rotation_angle) > 10:
    st.info(f"Applied {rotation_angle:.1f}° rotation normalization")

# STEP 2: CREATE FINAL OUTPUT IMAGE

final_output = image_rgb.copy()

# Compute Grad-CAM if enabled
gradcam_overlay = None
if enable_xai and bbox is not None:
    enabled_models = st.session_state.get('enabled_models',
list(MODELS.keys()))
    weights = st.session_state.get('model_weights',
DEFAULT_ENSEMBLE_WEIGHTS)
    accum = None
    total_w = 0.0

    for m_name in enabled_models:
        model = MODELS.get(m_name)
        if model is None:
            continue

        try:
            inp = preprocess_for_inference(best_rotation_crop, m_name)
            hm = make_gradcam_heatmap(inp, model)
            if hm is None:
                continue

            w = float(weights.get(m_name, 1.0))
            if accum is None:
                accum = hm * w
            else:
                accum = accum + hm * w
            total_w += w
        except Exception as e:
            logger.debug(f"Grad-CAM for {m_name} failed: {e}")

    # Create Grad-CAM overlay if we have a valid model-based heatmap
    # Note: If Grad-CAM is weak/unavailable, draw_full_rotated_overlay
will
    # automatically fall back to landmark-based heatmap (more
realistic)

    if accum is not None and total_w > 0:
        gradcam_map = accum / total_w
        gradcam_map = (gradcam_map - gradcam_map.min()) /

```

```

(gradcam_map.max() - gradcam_map.min() + 1e-9)
    gradcam_overlay = gradcam_map
else:
    # Set to None - landmark-based fallback will be used in
draw_full_rotated_overlay
    gradcam_overlay = None
    logger.debug("Grad-CAM unavailable, will use landmark-based
heatmap fallback.")

    # Apply overlays using the full-rotated overlay helper:
    # - If Grad-CAM is enabled, try to use real Grad-CAM, fallback to
landmark-based if weak
    # - If not enabled, we still rotate the full frame and draw box + text
only
    final_output = draw_full_rotated_overlay(
        image_rgb,
        bbox,
        landmarks,
        prediction,
        confidence,
        enable_xai, # Always pass user's enable_xai setting
        gradcam_overlay if enable_xai else None,
        rotation_applied,
        rotation_angle,
    )

    # Apply confusion pair analysis
    confusion_note = ""
    if st.session_state.get('show_confusion_analysis', True) and probs is
not None:
        _, _, confusion_note = handle_confusion_pairs(
            prediction, confidence, probs, model_votes,
            confusion_threshold=0.6
        )

    # Fallback warning
    if landmarks is None and used_fallback:
        st.caption("Detection used fallback crop (no MediaPipe landmarks).
Results may be less accurate.")

    # DISPLAY: Original + ONE Final Result

    with col2:
        caption_text = "Detection Result"
        if enable_xai and gradcam_overlay is not None:
            caption_text += " with Grad-CAM"
        st.image(final_output, caption=caption_text, use_column_width=True)

    st.markdown("---")

    # Show prediction info
    if confidence > conf_threshold:
        st.success(f"### Prediction: **{prediction}**")
    else:
        st.warning(f"### Prediction: **{prediction}** (Low confidence)")

    st.progress(float(confidence))
    st.caption(f"Confidence: {confidence:.1%}")

    if confusion_note:
        st.info(confusion_note)

    if use_tta:
        st.caption("TTA enabled - using multiple augmented views")

    # Model votes
    with st.expander("Model Votes", expanded=True):
        for name, vote in model_votes.items():
            st.write(f"**{name}**: {vote['class']}
({vote['confidence']:.1%})")

    # Top 5 predictions
    if probs is not None:
        with st.expander("Top 5 Predictions"):

```

```
top_5 = np.argsort(probs)[-5:][::-1]
for rank, idx in enumerate(top_5, 1):
    st.write(f"{rank}. **{CLASS_NAMES[idx]}**:")
{probs[idx]:.1%}")
else:
    st.error("No hand detected")
    st.info("Tips:\n- Ensure good lighting\n- Hand should be clearly
visible\n- Try different angles")
```


22. ✓ Tab 2: Video Upload

```
# TAB 2: VIDEO UPLOAD (FIXED)
def tab_video_upload():
    st.header("Video Upload")

    # Get settings from sidebar (tab-specific)
    conf_threshold = st.session_state.get('video_conf_threshold', 0.3)
    use_ensemble = st.session_state.get('video_use_ensemble', True)
    use_temporal_smoothing = st.session_state.get('video_temporal_smoothing', True)
    smoothing_window = st.session_state.get('smoothing_window', 5)
    preview_rate = st.session_state.get('preview_rate', 5)
    show_landmarks = st.session_state.get('video_show_landmarks', True)
    color_boxes = st.session_state.get('video_color_boxes', True)

    # Get fallback single model
    model_name = st.session_state.get('realtime_model', REALTIME_MODEL)
    model = MODELS.get(model_name)
    if model is None and len(MODELS) > 0:
        model_name = list(MODELS.keys())[0]
        model = MODELS[model_name]

    # Show mode info
    if use_ensemble and len(MODELS) > 1:
        num_models = len(MODELS)
        model_names = ', '.join(MODELS.keys()) if MODELS else 'None'
        st.info(f"***Ensemble Mode**": {num_models} models ({model_names})")
    else:
        st.info(f"***Single Model**": {model_name}")

    st.caption("Adjust settings in the sidebar for this tab")

    uploaded_video = st.file_uploader("Choose a video...", type=['mp4', 'avi', 'mov'],
key="video_uploader")

    if uploaded_video is not None:
        # Save uploaded video
        tfile = tempfile.NamedTemporaryFile(delete=False, suffix='.mp4')
        tfile.write(uploaded_video.read())
        video_path = tfile.name
        tfile.close()

        st.video(video_path)

        if st.button("🚀 Process Video", type="primary", use_container_width=True):
            with st.spinner("⌚ Processing video..."):
                cap = cv2.VideoCapture(video_path)
                if not cap.isOpened():
                    st.error("Failed to open video")
                    return

                # Video info
                fps = int(cap.get(cv2.CAP_PROP_FPS))
                total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
                width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
                height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

                # Output video
                if fps is None or fps <= 0 or not np.isfinite(fps):
                    fps = 25
                if width is None or width <= 0 or height is None or height <= 0:
                    width, height = (640, 480)

                output_path = tempfile.NamedTemporaryFile(delete=False, suffix='.mp4').name
                fourcc = cv2.VideoWriter_fourcc(*'mp4v')
                out = cv2.VideoWriter(output_path, fourcc, max(1, int(fps)), (int(width),
int(height)))

                # Create detector - match training: min_detection_confidence=0.6
                detector = mp.solutions.hands.Hands(
                    static_image_mode=False,
                    max_num_hands=1,
                    min_detection_confidence=0.6, # Match training exactly
                    min_tracking_confidence=0.5,
```



```

        model_complexity=1
    )

    # Progress tracking
    progress_bar = st.progress(0)
    status = st.empty()
    frame_preview = st.empty()
    predictions = []
    frame_count = 0
    detected = 0

    # Temporal smoothing for stable predictions (use sidebar setting)
    prob_history = deque(maxlen=smoothing_window)
    last_pred = "?"
    last_conf = 0.0

    while True:
        ret, frame_bgr = cap.read()
        if not ret:
            break

        frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
        frame_out_rgb = frame_rgb.copy()
        h, w = frame_rgb.shape[:2]

        # Direct MediaPipe detection for speed
        results = detector.process(frame_rgb)

        if results and results.multi_hand_landmarks:
            hand_landmarks = results.multi_hand_landmarks[0]
            # Get bounding box from landmarks
            x_coords = [lm.x for lm in hand_landmarks.landmark]
            y_coords = [lm.y for lm in hand_landmarks.landmark]
            x_min = min(x_coords)
            x_max = max(x_coords)
            y_min = min(y_coords)
            y_max = max(y_coords)

            # Add padding
            cx, cy = (x_min + x_max) / 2, (y_min + y_max) / 2
            side = max(x_max - x_min, y_max - y_min) * 1.4
            x1 = int(max(0, (cx - side/2) * w))
            y1 = int(max(0, (cy - side/2) * h))
            x2 = int(min(w, (cx + side/2) * w))
            y2 = int(min(h, (cy + side/2) * h))

            if x2 > x1 and y2 > y1:
                hand_crop = frame_rgb[y1:y2, x1:x2]
                if hand_crop.size > 0 and min(hand_crop.shape[:2]) > 20:
                    detected += 1

        # Prediction using ensemble or single model with temporal
smoothing
        try:
            if use_ensemble and len(MODELS) > 1:
                # Use ensemble prediction (all models)
                pred, conf, probs, _ = ensemble_predict(hand_crop,
conf_threshold)

                if probs is not None and use_temporal_smoothing:
                    prob_history.append(probs)
            else:
                # Single model prediction
                input_data = preprocess_for_inference(hand_crop,
model_name)

                output = model.predict(input_data, verbose=0)[0]
                probs = output / (output.sum() + 1e-9)
                if use_temporal_smoothing:
                    prob_history.append(probs)
                top_idx = int(np.argmax(probs))
                pred = CLASS_NAMES[top_idx]
                conf = float(probs[top_idx])

            # Apply temporal smoothing (if enabled)
            if use_temporal_smoothing and len(prob_history) >= 2:
                weights = np.array([0.1, 0.15, 0.2, 0.25, 0.3])[-

```

```

len(prob_history:]))

        weights = weights / weights.sum()
        smoothed = np.zeros_like(prob_history[0])
        for i, p in enumerate(prob_history):
            smoothed += p * weights[i]
        top_idx = int(np.argmax(smoothed))
        pred = CLASS_NAMES[top_idx]
        conf = float(smoothed[top_idx])

        last_pred = pred
        last_conf = conf
    except Exception as e:
        logger.debug(f"Predict error: {e}")
        pred = last_pred # Use last known prediction
        conf = last_conf

    if conf > conf_threshold:
        predictions.append((frame_count, pred, conf))

    # Draw on frame - color based on confidence (if enabled)
    if color_boxes:
        box_color = (0, 255, 0) if conf > 0.7 else (0, 255,
255) if conf > 0.4 else (0, 165, 255)
    else:
        box_color = (0, 255, 0)

    cv2.rectangle(frame_out_rgb, (x1, y1), (x2, y2), box_color,
2)

    # Add ensemble indicator if using ensemble
    num_models = len(MODELS) if use_ensemble else 1
    text = f"{pred}: {conf:.0%}" + (f" [{num_models}M]" if
use_ensemble and len(MODELS) > 1 else "")

    # Background for text readability
    (tw, th), _ = cv2.getTextSize(text,
cv2.FONT_HERSHEY_SIMPLEX, 0.7, 2)
    cv2.rectangle(frame_out_rgb, (x1, y1-25), (x1+tw+8, y1),
(0, 0, 0), -1)
    cv2.putText(frame_out_rgb, text, (x1+4, y1 - 6),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255),
2)

    # Draw landmarks (if enabled)
    if show_landmarks:
        mp_drawing = mp.solutions.drawing_utils
        mp_hands = mp.solutions.hands
        mp_drawing.draw_landmarks(
            frame_out_rgb, hand_landmarks,
mp_hands.HAND_CONNECTIONS,
thickness=2, circle_radius=2),
thickness=1)

        )

    # Update preview at configurable rate
    if frame_count % preview_rate == 0:
        frame_preview.image(frame_out_rgb, channels='RGB',
use_column_width=True)

    # Write frame to output video
    frame_out_bgr = cv2.cvtColor(frame_out_rgb, cv2.COLOR_RGB2BGR)
    try:
        frame_h, frame_w = frame_out_bgr.shape[:2]
        if (frame_w, frame_h) != (int(width), int(height)):
            frame_out_bgr = cv2.resize(frame_out_bgr, (int(width),
int(height)))
    except Exception:
        pass

    out.write(frame_out_bgr)

    frame_count += 1
    progress_bar.progress(min(frame_count / total_frames, 1.0))

```

```

        status.text(f"Frame {frame_count}/{total_frames} | Detected:
{detected}")

    detector.close()
    cap.release()
    out.release()

    st.success("Processing complete!")

    # Stats
    col_a, col_b = st.columns(2)
    with col_a:
        st.metric("Total Frames", total_frames)
    with col_b:
        st.metric("Hands Detected", detected)

    # Show output video
    if detected > 0:
        st.video(output_path)

    # Sign statistics
    if predictions:
        st.markdown("### 📊 Detected Signs")
        sign_counts = Counter([p[1] for p in predictions])
        sign_confs = {}
        for _, pred, conf in predictions:
            if pred not in sign_confs:
                sign_confs[pred] = []
            sign_confs[pred].append(conf)

        for sign, count in sign_counts.most_common():
            avg_conf = np.mean(sign_confs[sign])
            st.write(f"**{sign}**: {count} times | Avg confidence:
{avg_conf:.1%}")
    else:
        st.warning(f"⚠️ No hands detected. Try:\n- Better lighting\n- Clearer
hand visibility")

    os.unlink(video_path)

```

23. ✓ Tab 4: Live Camera (WebRTC)

```
# TAB 4: LIVE CAMERA (WebRTC)
def tab_live_camera_webrtc():
    st.header("Live Camera 📹 Real-time ASL Recognition")

    # Get settings from sidebar (tab-specific)
    use_ensemble = st.session_state.get('live_use_ensemble', True)
    single_model_name = st.session_state.get('live_single_model', REALTIME_MODEL)
    skip_frames = st.session_state.get('live_skip_frames', 2)
    smoothing_window = st.session_state.get('live_smoothing', 5)
    mp_detection_conf = st.session_state.get('mp_detection_conf', 0.6)
    mp_tracking_conf = st.session_state.get('mp_tracking_conf', 0.5)
    crop_padding = st.session_state.get('crop_padding', 1.4)
    show_landmarks = st.session_state.get('live_show_landmarks', True)
    show_fps = st.session_state.get('live_show_fps', False)
    color_boxes = st.session_state.get('live_color_boxes', True)
    conf_threshold = st.session_state.get('live_conf_threshold', 0.3)

    # Get primary model for single-model mode
    model = MODELS.get(single_model_name)
    if model is None and len(MODELS) > 0:
        single_model_name = list(MODELS.keys())[0]
        model = MODELS[single_model_name]

    # Display mode info
    if use_ensemble:
        num_models = len(MODELS)
        model_names = ', '.join(MODELS.keys()) if MODELS else 'None'
        st.info(f"***Ensemble Mode***: Using {num_models} models ({model_names})")
    else:
        st.info(f"***Single Model***: {single_model_name}")

    # Store settings in a mutable container that the class can access
    settings_ref = {
        'models': MODELS,
        'single_model': model,
        'single_model_name': single_model_name,
        'use_ensemble': use_ensemble,
        'classes': CLASS_NAMES,
        'weights': DEFAULT_ENSEMBLE_WEIGHTS,
        'skip_frames': skip_frames,
        'smoothing_window': smoothing_window,
        'mp_detection_conf': mp_detection_conf,
        'mp_tracking_conf': mp_tracking_conf,
        'crop_padding': crop_padding,
        'show_landmarks': show_landmarks,
        'show_fps': show_fps,
        'color_boxes': color_boxes,
        'conf_threshold': conf_threshold,
    }

class ASLVideoProcessor(VideoTransformerBase):
    """
    Real-time video processor with configurable ENSEMBLE or SINGLE MODEL mode.
    Features:
    - Toggle between ensemble (all models) or single model
    - Weighted ensemble voting for better accuracy
    - Temporal smoothing for stable predictions
    - Configurable skip frames for speed optimization
    """
    def __init__(self):
        # Get settings from outer scope
        s = settings_ref

        # Create MediaPipe detector with configurable confidence
        self.detector = mp.solutions.hands.Hands(
            static_image_mode=False,
            max_num_hands=1,
            min_detection_confidence=s['mp_detection_conf'],
            min_tracking_confidence=s['mp_tracking_conf'],
            model_complexity=1
        )
```

```

self.frame_count = 0
self.last_pred = "?"
self.last_conf = 0.0
self.last_time = time.time()
self.fps = 0.0

# Model settings
self.models = s['models']
self.single_model = s['single_model']
self.single_model_name = s['single_model_name']
self.use_ensemble = s['use_ensemble']
self.class_names = s['classes']
self.weights = s['weights']

# Display settings
self.crop_padding = s['crop_padding']
self.show_landmarks = s['show_landmarks']
self.show_fps = s['show_fps']
self.color_boxes = s['color_boxes']
self.conf_threshold = s['conf_threshold']

# Temporal smoothing: store last N predictions
self.prediction_history = deque(maxlen=s['smoothing_window'])
self.prob_history = deque(maxlen=s['smoothing_window'])

# Frame skipping for speed
self.predict_every_n = s['skip_frames']
self.last_ensemble_probs = None

def __del__(self):
    try:
        if self.detector:
            self.detector.close()
    except:
        pass

def recv(self, frame: av.VideoFrame) -> av.VideoFrame:
    """Process frame with configurable ensemble/single model mode."""
    self.frame_count += 1
    img_bgr = frame.to_ndarray(format="bgr24")

    # Calculate FPS
    current_time = time.time()
    if self.frame_count % 10 == 0:
        self.fps = 10.0 / max(0.001, current_time - self.last_time)
        self.last_time = current_time

    try:
        img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
        h, w = img_rgb.shape[:2]

        # MediaPipe detection
        results = self.detector.process(img_rgb)
        out_rgb = img_rgb.copy()

        # Show FPS if enabled
        if self.show_fps:
            cv2.putText(out_rgb, f"FPS: {self.fps:.1f}", (w - 100, 25),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

        if results and results.multi_hand_landmarks:
            hand_landmarks = results.multi_hand_landmarks[0]
            # Get bounding box from landmarks
            x_coords = [lm.x for lm in hand_landmarks.landmark]
            y_coords = [lm.y for lm in hand_landmarks.landmark]
            x_min = min(x_coords)
            x_max = max(x_coords)
            y_min = min(y_coords)
            y_max = max(y_coords)

            # Add configurable padding
            cx, cy = (x_min + x_max) / 2, (y_min + y_max) / 2
            side = max(x_max - x_min, y_max - y_min) * self.crop_padding
            x1 = int(max(0, (cx - side/2) * w))
            y1 = int(max(0, (cy - side/2) * h))

```

```

x2 = int(min(w, (cx + side/2) * w))
y2 = int(min(h, (cy + side/2) * h))

if x2 > x1 and y2 > y1:
    hand_crop = img_rgb[y1:y2, x1:x2]
    if hand_crop.size > 0 and min(hand_crop.shape[:2]) > 20:
        # Skip frames for speed - use cached prediction
        if self.frame_count % self.predict_every_n == 0:
            if self.use_ensemble and len(self.models) > 1:
                # Run ensemble prediction
                pred, conf = self._ensemble_predict(hand_crop)
            else:
                # Run single model prediction
                pred, conf = self._single_predict(hand_crop)

            self.last_pred = pred
            self.last_conf = conf

        pred, conf = self.last_pred, self.last_conf

    # Draw bounding box - color based on confidence (if enabled)
    if self.color_boxes:
        box_color = (0, 255, 0) if conf > 0.7 else (0, 255, 255) if
conf > 0.4 else (0, 165, 255)
    else:
        box_color = (0, 255, 0)

    cv2.rectangle(out_rgb, (x1, y1), (x2, y2), box_color, 2)

    # Show prediction info
    if self.use_ensemble:
        num_models = len(self.models)
        text = f"{pred}: {conf:.0%} [{num_models}M]"
    else:
        text = f"{pred}: {conf:.0%}"

    # Background for text
    (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX,
0.8, 2)
    cv2.rectangle(out_rgb, (x1, y1-30), (x1+tw+10, y1), (0, 0, 0),
-1)
    cv2.putText(out_rgb, text, (x1+5, y1 - 8),
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

    # Draw landmarks (if enabled)
    if self.show_landmarks:
        mp_drawing = mp.solutions.drawing_utils
        mp_hands = mp.solutions.hands
        mp_drawing.draw_landmarks(
            out_rgb, hand_landmarks, mp_hands.HAND_CONNECTIONS,
            mp_drawing.DrawingSpec(color=(0, 255, 0), thickness=2,
circle_radius=2),
            mp_drawing.DrawingSpec(color=(255, 0, 0), thickness=1)
        )
    else:
        # No detection - show last prediction with "waiting" indicator
        cv2.putText(out_rgb, f"Last: {self.last_pred} (waiting...)", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (200, 200, 0), 2)

    out_bgr = cv2.cvtColor(out_rgb, cv2.COLOR_RGB2BGR)
    return av.VideoFrame.from_ndarray(out_bgr, format="bgr24")
except Exception as e:
    logger.debug(f"Frame error: {e}")
    return frame

def _preprocess_for_model(self, img: np.ndarray, model_name: str) -> np.ndarray:
    """Apply correct preprocessing for each model."""
    img_float = img.astype(np.float32)
    if model_name == 'EfficientNetV2B3' and effnet_preprocess is not None:
        return effnet_preprocess(img_float)
    elif model_name == 'ResNet50' and resnet_preprocess is not None:
        return resnet_preprocess(img_float)
    elif model_name == 'InceptionV3' and inception_preprocess is not None:
        return inception_preprocess(img_float)
    else:

```



```

        return img_float / 255.0

def _single_predict(self, hand_crop: np.ndarray) -> Tuple[str, float]:
    """
    Single model prediction with temporal smoothing.
    """
    try:
        if self.single_model is None:
            return "No Model", 0.0

        # Resize
        img = cv2.resize(hand_crop, (IMG_SIZE, IMG_SIZE))

        # Preprocess for single model
        img_preprocessed = self._preprocess_for_model(img, self.single_model_name)
        img_batch = np.expand_dims(img_preprocessed, axis=0)

        # Predict
        output = self.single_model.predict(img_batch, verbose=0)[0]
        probs = output / (output.sum() + 1e-9)

        # Add to history for temporal smoothing
        self.prob_history.append(probs)

        # Temporal smoothing
        if len(self.prob_history) >= 2:
            weights = np.array([0.1, 0.15, 0.2, 0.25, 0.3][len(self.prob_history):])
            weights = weights / weights.sum()
            smoothed = np.zeros_like(probs)
            for i, p in enumerate(self.prob_history):
                smoothed += p * weights[i]
            probs = smoothed

        top_idx = int(np.argmax(probs))
        return self.class_names[top_idx], float(probs[top_idx])
    except Exception as e:
        logger.debug(f"Single predict error: {e}")
        return "Error", 0.0

def _ensemble_predict(self, hand_crop: np.ndarray) -> Tuple[str, float]:
    """
    Ensemble prediction using ALL available models with weighted voting.
    Returns smoothed prediction for stability.
    """
    try:
        if not self.models:
            return "No Models", 0.0

        # Resize once
        img = cv2.resize(hand_crop, (IMG_SIZE, IMG_SIZE))

        # Collect weighted probabilities from all models
        ensemble_probs = np.zeros(len(self.class_names), dtype=np.float32)
        total_weight = 0.0

        for model_name, model in self.models.items():
            if model is None:
                continue

            try:
                # Preprocess for this specific model
                img_preprocessed = self._preprocess_for_model(img, model_name)
                img_batch = np.expand_dims(img_preprocessed, axis=0)

                # Get prediction
                output = model.predict(img_batch, verbose=0)[0]
                probs = output / (output.sum() + 1e-9)

                # Apply weight
                weight = self.weights.get(model_name, 0.25)
                ensemble_probs += probs * weight
                total_weight += weight
            except Exception as e:
                logger.debug(f"Model {model_name} prediction failed: {e}")

```



```

        continue

    if total_weight == 0:
        return "Error", 0.0

    # Normalize by total weight
    ensemble_probs /= total_weight

    # Add to history for temporal smoothing
    self.prob_history.append(ensemble_probs)

    # Temporal smoothing: average over recent predictions
    if len(self.prob_history) >= 2:
        # Weighted average favoring recent predictions
        smoothed = np.zeros_like(ensemble_probs)
        weights = np.array([0.1, 0.15, 0.2, 0.25, 0.3][-
len(self.prob_history):])
        weights = weights / weights.sum()
        for i, p in enumerate(self.prob_history):
            smoothed += p * weights[i]
        ensemble_probs = smoothed

    # Get final prediction
    top_idx = int(np.argmax(ensemble_probs))
    confidence = float(ensemble_probs[top_idx])
    prediction = self.class_names[top_idx]

    # Store in history
    self.prediction_history.append(prediction)
    self.last_ensemble_probs = ensemble_probs

    return prediction, confidence
except Exception as e:
    logger.debug(f"Ensemble predict error: {e}")
    return "Error", 0.0

try:
    webrtc_streamer(
        key="asl-live-webrtc",
        mode=WebRtcMode.SENDRECV,
        video_processor_factory=ASLVideoProcessor,
        media_stream_constraints={
            "video": {
                "width": {"ideal": 640},
                "height": {"ideal": 480},
                "frameRate": {"ideal": 30}
            },
            "audio": False
        },
        async_processing=True,
        rtc_configuration={
            "iceServers": [{"urls": ["stun:stun.l.google.com:19302"]}]}
    )

    # Settings info is shown at top, no need to duplicate here
    st.caption("💡 Adjust settings in the sidebar for this tab")
except Exception as e:
    logger.error(f"webrtc_streamer failed: {e}")
    st.error("⚠️ Live camera failed to start.")
    st.info("Try using the Image or Video Upload tabs instead.")

```

24. ✓ Tab 4: Model Comparison

```
# TAB 4: MODEL COMPARISON
def tab_model_comparison():
    """Display model comparison, architecture details, and performance metrics."""
    st.header("Model Comparison & Analysis")

    # Model Status Section
    st.subheader("Model Status")
    available_models = list(MODELS.keys())
    col1, col2, col3 = st.columns(3)
    for idx, model_name in enumerate(available_models):
        model = MODELS.get(model_name)
        with [col1, col2, col3][idx % 3]:
            if model is not None:
                st.success(f"{model_name}")
            else:
                st.error(f"{model_name}")

    st.markdown("----")

    st.markdown("""
    This tab shows the comparison between the 3 CNN architectures used in this project:
    - **ResNet50**: Deep residual learning with skip connections
    - **InceptionV3**: Multi-scale feature extraction with inception modules
    - **EfficientNetV2B3**: Compound scaling with neural architecture search
    """)

    # Architecture Comparison Table
    st.subheader("Architecture Comparison")
    arch_data = {
        'Model': ['ResNet50', 'InceptionV3', 'EfficientNetV2B3'],
        'Parameters (M)': ['~25.6', '~23.8', '~14.4'],
        'Top Layers Unfrozen': ['30', '60', '80'],
        'Input Size': ['224x224', '224x224', '224x224'],
        'Key Innovation': ['Skip Connections', 'Inception Modules', 'Compound Scaling'],
        'Best For': ['General Features', 'Multi-Scale Patterns', 'Efficiency + Accuracy']
    }
    import pandas as pd
    df_arch = pd.DataFrame(arch_data)
    st.dataframe(df_arch, use_container_width=True, hide_index=True)

    # Model Details
    st.subheader("Loaded Model Details")
    col1, col2, col3 = st.columns(3)
    for idx, (name, model) in enumerate(MODELS.items()):
        with [col1, col2, col3][idx % 3]:
            st.markdown(f"***{name}***")
            if model is not None:
                try:
                    total_params = model.count_params()
                    trainable = sum([tf.size(w).numpy() for w in model.trainable_weights])
                    st.metric("Total Parameters", f"{total_params:,}")
                    st.metric("Trainable", f"{trainable:,}")
                    st.success("Loaded")
                except Exception as e:
                    st.warning(f"Could not get params: {e}")
            else:
                st.error("Not loaded")

    st.markdown("----")

    # Training Information
    st.subheader("Training Configuration")
    train_info = {
        'Setting': ['Images per Class', 'Image Size', 'Batch Size', 'Epochs',
                   'Validation Split', 'MediaPipe Confidence', 'Augmentation'],
        'Value': ['1000', '224x224', '32', '35', '15%', '0.6', 'Albumentations (model-specific)']
    }
    df_train = pd.DataFrame(train_info)
    st.dataframe(df_train, use_container_width=True, hide_index=True)

    st.markdown("----")
```

```

# Ensemble Weights
st.subheader("Ensemble Weights")
st.markdown("The ensemble prediction uses weighted averaging of model outputs
(research-optimized):")
weights = st.session_state.get('model_weights', DEFAULT_ENSEMBLE_WEIGHTS)
for name, weight in weights.items():
    st.progress(float(weight), text=f"{name}: {weight:.0%}")

st.markdown("---")

# Confusion Matrix Visualization (if available)
st.subheader("Class Confusion Analysis")
st.markdown("""
**Common Confusions in ASL Alphabet:**
- **Similar Hand Shapes**: M/N/S, A/E/S, D/K, U/V/R
- **Orientation-Dependent**: K (rotated), P (rotated L), H (sideways U)
- **Finger-based**: I/J (same shape, J has motion), Z (motion-based)
""")

# Show a sample confusion matrix heatmap for expected confusions
st.markdown("#### Expected Class Similarities")
similar_classes = [
    ("M", "N", "Similar fist shape"),
    ("A", "S", "Closed fist variants"),
    ("D", "K", "Index finger pointing"),
    ("U", "V", "Two fingers up"),
    ("P", "K", "P is rotated K"),
    ("H", "U", "H is sideways U"),
]
for c1, c2, reason in similar_classes:
    st.write(f"🔵 **{c1} ↔ {c2}**: {reason}")

```

25. ✓ Main Application Logic

```
# MAIN APP
def render_sidebar_settings():
    """Render tab-specific sidebar settings only."""
    active_tab = st.session_state.get('active_tab', 'Image')
    available_models = list(MODELS.keys())

    # ===== TAB-SPECIFIC SETTINGS =====
    # ----- IMAGE TAB SETTINGS -----
    if active_tab == 'Image':
        st.markdown("### Image Settings")

        # Model Selection
        st.markdown("#### Model Selection")
        # Enable/disable individual models for ensemble
        st.markdown("**Enabled Models:**")
        enabled_models = []
        for model_name in available_models:
            if st.checkbox(model_name, value=True, key=f'enable_{model_name}'):
                enabled_models.append(model_name)
        st.session_state['enabled_models'] = enabled_models if enabled_models else
available_models

        # Custom weights
        if st.checkbox("Custom Weights", value=False, key='use_custom_weights'):
            st.markdown("**Adjust Weights:**")
            custom_weights = {}
            for model_name in available_models:
                weight = st.slider(
                    f"{model_name}",
                    min_value=0.0, max_value=1.0,
                    value=DEFAULT_ENSEMBLE_WEIGHTS.get(model_name, 0.33),
                    step=0.05,
                    key=f'weight_{model_name}'
                )
                custom_weights[model_name] = weight

            # Normalize
            total = sum(custom_weights.values())
            if total > 0:
                custom_weights = {k: v/total for k, v in custom_weights.items()}
            st.session_state['model_weights'] = custom_weights
        else:
            st.session_state['model_weights'] = DEFAULT_ENSEMBLE_WEIGHTS

        st.markdown("---")

        # Accuracy Enhancement
        st.markdown("#### Accuracy")
        st.checkbox(
            "Enable TTA (Test-Time Augmentation)",
            value=st.session_state.get('enable_tta', True),
            key='enable_tta',
            help="Use multiple augmented views for better accuracy"
        )
        st.checkbox(
            "Show Confusion Analysis",
            value=st.session_state.get('show_confusion_analysis', True),
            key='show_confusion_analysis',
            help="Analyze commonly confused letters"
        )
        st.checkbox(
            "Show Top-5 Predictions",
            value=st.session_state.get('show_top5', True),
            key='show_top5',
            help="Display top 5 predictions with probabilities"
        )
        st.markdown("---")

        # Grad-CAM toggle only (no other XAI modes)
        st.markdown("#### Grad-CAM")
        st.checkbox(
            "Enable Grad-CAM",
```

```

        value=st.session_state.get('enable_xai', False),
        key='enable_xai',
        help="Highlight regions the model uses for its decision"
    )

# ----- VIDEO TAB SETTINGS -----
elif active_tab == 'Video':
    # Model Mode
    st.markdown("#### Model Mode")
    video_mode = st.radio(
        "Prediction Mode",
        ["Ensemble (All Models)", "Single Model (Faster)"],
        index=0 if st.session_state.get('video_use_ensemble', True) else 1,
        key='video_mode_radio',
        help="Ensemble is more accurate, Single is faster"
    )
    st.session_state['video_use_ensemble'] = video_mode.startswith("Ensemble")

    # Single model selection (when ensemble disabled)
    if not st.session_state.get('video_use_ensemble', True):
        if available_models:
            default_idx = available_models.index(REALTIME_MODEL) if REALTIME_MODEL in
available_models else 0
            st.selectbox(
                "Select Model",
                available_models,
                index=default_idx,
                key='realtime_model',
                help="Single model for video processing"
            )

        st.markdown("---")

    # Video Processing Options
    st.markdown("#### Processing Options")
    st.checkbox(
        "Temporal Smoothing",
        value=st.session_state.get('video_temporal_smoothing', True),
        key='video_temporal_smoothing',
        help="Smooth predictions over time for stability"
    )
    st.slider(
        "Smoothing Window",
        min_value=2, max_value=10,
        value=st.session_state.get('smoothing_window', 5),
        key='smoothing_window',
        help="Frames for temporal averaging"
    )
    st.slider(
        "Preview Rate",
        min_value=1, max_value=15,
        value=st.session_state.get('preview_rate', 5),
        key='preview_rate',
        help="Update preview every N frames"
    )
    st.markdown("---")

    # Display Options
    st.markdown("#### Display")
    st.checkbox(
        "Show Landmarks",
        value=st.session_state.get('video_show_landmarks', True),
        key='video_show_landmarks'
    )
    st.checkbox(
        "Color-Coded Boxes",
        value=st.session_state.get('video_color_boxes', True),
        key='video_color_boxes',
        help="Green>70%, Yellow>40%, Orange<40%"
    )

# ----- LIVE CAMERA TAB SETTINGS -----
elif active_tab == 'Live Camera':
    # Model Mode Selection
    st.markdown("#### Model Mode")

```

```

live_mode = st.radio(
    "Prediction Mode",
    ["Ensemble (All Models)", "Single Model (Faster)"],
    index=0 if st.session_state.get('live_use_ensemble', True) else 1,
    key='live_mode_radio',
    help="Ensemble is more accurate, Single is faster"
)
st.session_state['live_use_ensemble'] = live_mode.startswith("Ensemble")

# Single model selection
if not st.session_state.get('live_use_ensemble', True):
    if available_models:
        default_idx = available_models.index(REALTIME_MODEL) if REALTIME_MODEL in
available_models else 0
        st.selectbox(
            "Select Model",
            available_models,
            index=default_idx,
            key='live_single_model'
        )

st.markdown("---")

# Performance Options
st.markdown("#### Performance")
st.slider(
    "Skip Frames",
    min_value=1, max_value=5,
    value=st.session_state.get('live_skip_frames', 2),
    key='live_skip_frames',
    help="Predict every N frames (higher = faster)"
)
st.slider(
    "Smoothing Window",
    min_value=2, max_value=10,
    value=st.session_state.get('live_smoothing', 5),
    key='live_smoothing',
    help="Frames for temporal smoothing"
)
st.markdown("---")

# Detection Settings
st.markdown("#### Hand Detection")
st.slider(
    "Detection Confidence",
    min_value=0.3, max_value=0.9,
    value=st.session_state.get('mp_detection_conf', 0.6),
    step=0.1,
    key='mp_detection_conf',
    help="MediaPipe minimum detection confidence"
)
st.slider(
    "Tracking Confidence",
    min_value=0.3, max_value=0.9,
    value=st.session_state.get('mp_tracking_conf', 0.5),
    step=0.1,
    key='mp_tracking_conf',
    help="MediaPipe tracking confidence"
)
st.slider(
    "Crop Padding",
    min_value=1.0, max_value=2.0,
    value=st.session_state.get('crop_padding', 1.4),
    step=0.1,
    key='crop_padding',
    help="Hand crop padding factor"
)
st.markdown("---")

# Display Options
st.markdown("#### Display")
st.checkbox(
    "Show Hand Landmarks",
    value=st.session_state.get('live_show_landmarks', True),
    key='live_show_landmarks'
)

```



```

    )
    st.checkbox(
        "Show FPS Counter",
        value=st.session_state.get('live_show_fps', False),
        key='live_show_fps'
    )
    st.checkbox(
        "Color-Coded Boxes",
        value=st.session_state.get('live_color_boxes', True),
        key='live_color_boxes'
    )

# ----- COMPARISON TAB SETTINGS -----
elif active_tab == 'Comparison':
    st.checkbox(
        "Show Architecture Details",
        value=st.session_state.get('show_arch_details', True),
        key='show_arch_details'
    )
    st.checkbox(
        "Show Training Info",
        value=st.session_state.get('show_training_info', True),
        key='show_training_info'
    )
    st.checkbox(
        "Show Confusion Analysis",
        value=st.session_state.get('show_confusion_info', True),
        key='show_confusion_info'
    )

def main():
    st.title("ASL Multi-Model Classifier")
    st.markdown("**Ensemble AI System with Real Keras Models**")
    st.markdown("---")

    # Use sidebar for tab selection to ensure proper state management
    with st.sidebar:
        st.markdown("## Navigation")
        tab_options = ['Image', 'Video', 'Live Camera', 'Model Comparison']
        tab_keys = ['Image', 'Video', 'Live Camera', 'Comparison']

        # Initialize active_tab if not present
        if 'active_tab' not in st.session_state:
            st.session_state['active_tab'] = 'Image'

        # Get current index based on active_tab
        try:
            current_idx = tab_keys.index(st.session_state['active_tab'])
        except ValueError:
            current_idx = 0

        # Tab selection via radio buttons in sidebar
        # Use the radio widget's value directly to sync with session state
        selected_idx = st.radio(
            "Select Tab",
            range(len(tab_options)),
            format_func=lambda x: tab_options[x],
            index=current_idx,
            key='tab_selector',
            label_visibility="collapsed"
        )

        # Update active tab immediately based on selection
        st.session_state['active_tab'] = tab_keys[selected_idx]

        st.markdown("---")

        # Render tab-specific settings
        render_sidebar_settings()

    # Render the selected tab content in main area
    active = st.session_state.get('active_tab', 'Image')
    if active == 'Image':
        tab_image_upload()
    elif active == 'Video':

```



```
        tab_video_upload()
    elif active == 'Live Camera':
        tab_live_camera_webrtc()
    elif active == 'Comparison':
        tab_model_comparison()

if __name__ == "__main__":
    main()
```